WAZUH SOC ANALYST HOME LAB — 14-DAY INTENSIVE PROGRAM

PowerShell Detection Lab

Detecting and investigating suspicious PowerShell activity with Windows logging, Sysmon, Wazuh and Splunk

Author	Aigbokhaode Hope Imomoh (Hope), SOC Analyst Trainee, Abuja, Nigeria
Report date	6 November 2026 | Version 1.0
Lab period	3 – 6 November 2026 (screenshots show local time, UTC+1)
Platform	VirtualBox lab: Windows 10 VM (monitored) and Ubuntu VM (Wazuh + Splunk)
Purpose	Portfolio, junior SOC analyst application, training and assessment
Evidence status	Every result in this report is tied to a screenshot or photo taken during the lab. Items that were not captured are marked as gaps, not as successes.

How to read this report. Non-technical readers: read the one-page summary that follows and the boxes marked **IN PLAIN ENGLISH**. Beginners who want to repeat the lab: go straight to Chapter 15 (Reproduction Guide). Reviewers who want proof: see Chapter 9 (Evidence) and Appendix A (Evidence Register).

The whole lab in one page (no technical knowledge needed)

The problem. Every Windows computer comes with a powerful tool called **PowerShell**. System administrators use it to manage computers. Attackers love it for the same reason: it is already installed, it is trusted, and it can do almost anything. Attackers also try to hide what they type, for example by scrambling the command into unreadable text or by running it in a window nobody can see. Out of the box, Windows keeps very little record of what was typed.

The analogy. Think of a shop. PowerShell is a master key that opens every door. This lab is about fitting the shop with cameras, a recorder, a control room and an alarm:

- **Cameras** = PowerShell logging (we switched it on so every command is written down).
- **The recorder with faces and timestamps** = Sysmon (it records who started which program, and what started it).
- **The control room** = Wazuh (it collects everything in one place and watches for suspicious patterns).
- **The alarm we wrote ourselves** = a custom Wazuh rule that triggers when PowerShell is started with a hidden window.
- **A second control room** = Splunk (we cross-checked that it saw exactly the same events).

What we did. We turned the cameras on, checked the recorder was working, connected both to the control room, and then deliberately ran three *harmless* commands that look suspicious (they only displayed the user name, computer name and running programs). We then confirmed the alarms went off.

What we found.

- The control room raised **two high-level alerts (level 12)** for the scrambled (encoded) command, and a lower-level alert (level 4) for the program-listing command.
- The alarm we wrote ourselves **went off once** when we ran the hidden-window command (level 10).
- Splunk showed the **same five suspicious program launches** and the same script text, so two separate tools agree.
- Along the way the recording computer **ran low on disk space** and Splunk refused to search until we freed space. That is a real-world lesson: monitoring tools need housekeeping too.

> **VERIFIED RESULT**
> The lab met all four required deliverables: a PowerShell alert, a Sysmon correlation, a custom rule and a MITRE ATT&CK mapping, with evidence. Two things could not be proven and are reported as such: Splunk could not show the process ID, and we did not run a harmless “normal admin” hidden-window command to measure false alarms.

Why it matters. A SOC (Security Operations Centre) analyst’s daily job is exactly this loop: collect evidence, spot patterns, investigate, decide. This report documents that loop end to end, with enough detail that a beginner can repeat it.

Table of Contents

The whole lab in one page (no technical knowledge needed) 2

Table of Contents 3

List of Figures 6

List of Tables 7

Abbreviations and Acronyms 8

Chapter 1: Executive Summary 9

1.1 Purpose and problem . . . 9

1.2 Method in six lines . . . 9

1.3 Verified outcomes . . . 9

1.4 Not proven . . . 9

Chapter 2: Introduction and Objectives 11

2.1 Background and motivation . . . 11

2.2 Scope . . . 11

2.3 Objectives and success criteria . . . 11

Chapter 3: Technical Background 13

3.1 The detection pipeline . . . 13

3.2 Key concepts . . . 13

Chapter 4: Lab Environment and Architecture 15

4.1 How the parts talk to each other . . . 15

Chapter 5: Prerequisites and Safety 16

5.1 What you need before starting . . . 16

5.2 Safety rules . . . 16

5.3 Restoring if something goes wrong . . . 16

Chapter 6: Step-by-Step Implementation 17

Step 1 – Turn on PowerShell logging . . . 17

Step 2 – Confirm the recorder (Sysmon) and agent (Wazuh) are running . . . 18

Step 3 – Tell the Wazuh agent to collect the two log channels . . . 18

Step 4 – Prove the manager receives the events . . . 20

Step 5 – Generate the test activity . . . 20

Step 6 – Read the Sysmon process context locally . . . 21
Step 7 – Find the built-in Wazuh alerts . . . 23
Step 8 – Write and test the custom rule . . . 23
Step 9 – Cross-check in Splunk . . . 23

Chapter 7: Configuration and Detection Logic 25

7.1 Original configuration versus changes . . . 25
7.2 The custom rule, line by line . . . 25
7.3 Why these conditions, and what they miss . . . 25
7.4 Queries used . . . 26

Chapter 8: Testing and Results 27

8.1 Test results . . . 27
8.2 Timeline of the lab . . . 27
8.3 The six analyst questions from the brief . . . 28

Chapter 9: Evidence and Screenshot Analysis 29

9.1 Built-in alert: encoded PowerShell command (rule 92057) . . . 29
9.2 PowerShell script block (4104) and rule 91815 . . . 31
9.3 Custom rule 100600 . . . 33
9.4 Splunk cross-check . . . 34

Chapter 10: Troubleshooting and Root-Cause Analysis 38

10.1 Issue 4: a harmless typo . . . 38
10.2 Issue 2: Sysmon service name . . . 39
10.3 Issue 5: the typo in Splunk . . . 39
10.4 Issue 7: disk full . . . 40
10.5 Issue 8: field not extracted . . . 41

Chapter 11: Security Findings and Risk Analysis 43

11.1 Findings . . . 43
11.2 False-positive considerations (brief requirement) . . . 43

Chapter 12: Mitigation and Remediation 44

Chapter 13: What I Learned 45

Chapter 14: Practical Knowledge and Competencies 46

Chapter 15: Reproduction Guide (for a beginner) 47

15.1 Starting state . . . 47
15.2 Enable PowerShell logging (Windows 10, PowerShell as Administrator) . . . 47

15.3 Check Sysmon and the agent (Windows 10) . . . 47
15.4 Collect the logs (Windows 10) . . . 47
15.5 Optional: archive logging (Ubuntu VM) . . . 47
15.6 Generate the test events (Windows 10) . . . 48
15.7 Find the alerts and add your own rule . . . 48
15.8 Troubleshooting quick guide . . . 48
15.9 Cleanup and rollback . . . 48

Chapter 16: Limitations and Future Improvements **50**

Future improvements . . . 50

Chapter 17: Conclusion **51**

Appendix A: Evidence Register **52**

Appendix B: Commands Used **54**

Appendix C: Splunk Queries **55**

Appendix D: Glossary in Plain Language **56**

Appendix E: Integrity and Redaction Note **57**

List of Figures

Figure 1 – The Day 6 lab brief (source of the tasks, deliverables and analyst questions; also the layout reference for this report) 11

Figure 2 – The six stages from event to investigation. A problem at any stage looks like “no alert”. 13

Figure 3 – Laboratory architecture and data flow (drawn from the IP addresses, agent ID and index names seen in the evidence) 15

Figure 4 – The registry keys being created (Windows 10, 3 Nov 2026, 12:57 PM) 17

Figure 5 – ModuleNames set to “*” and read back, completing module logging 18

Figure 6 – Event ID 4104 entries showing the commands just typed (12:49–12:55 PM) 18

Figure 7 – Sysmon found running under the service name “Sysmon” 18

Figure 8 – ossec.conf in Notepad: the existing Sysmon block (5 Nov 2026, 2:48 PM) 19

Figure 9 – Agent restarted and reported Running (2:55 PM) 19

Figure 10 – Select-String shows both channels at lines 50 and 65 (2:59 PM). The two red sudo errors are explained in Chapter 10, Issue 3. 20

Figure 11 – Archive entry for agent DESKTOP-T6T1N2V, channel PowerShell/Operational, Event ID 4104; logall and logall_json set to yes 20

Figure 12 – Sysmon Event 1: encoded PowerShell (PID 9492) started at 3:07:03 PM by parent PID 7240 21

Figure 13 – Sysmon Event 1: the earlier encoded run (PID 6724) at 3:05:40 PM, parent PID 7672 22

Figure 14 – Child processes whoami.exe and HOSTNAME.EXE; their ParentCommandLine is the encoded command 22

Figure 15 – Sysmon Event 1: hidden-window PowerShell (PID 2444) at 3:07:04 PM, parent PID 7240 22

Figure 16 – Wazuh Threat Hunting results for the encoded-command query: two level-12 alerts (6 Nov 2026) 29

Figure 17 – Alert fields: the command line as seen by Wazuh 30

Figure 18 – Alert fields: parent process and process IDs 30

Figure 19 – Alert fields: rule metadata for 92057 31

Figure 20 – Wazuh results for 4104 script-block events: rule 91815, level 4 31

Figure 21 – The 4104 event: script text and PowerShell process ID 32

Figure 22 – The same 4104 event: record number and script-block ID 32

Figure 23 – Rule 91815 metadata 33

Figure 24 – Wazuh results for rule.id: 100600 (6 Nov 2026, 08:45) 33

Figure 25 – The alert’s command line: hidden window 34

Figure 26 – Custom rule metadata and MITRE mapping 34

Figure 27 – Splunk: which data exists for the Windows host (6 Nov 2026, 09:28) 35

Figure 28 – Splunk: five PowerShell Sysmon events (time, host, user, parent, command line) 35

Figure 29 – The same table scrolled right: command lines and ProcessGuids 36

Figure 30 – Splunk: the three hidden-window runs found with a simple keyword search 36

Figure 31 – Splunk raw 4104 event (15:07:03): record number 1889 and script-block ID 37

Figure 32 – Splunk 4104 search: six matching events (photo of screen, 6 Nov 10:26) 37

Figure 33 – Session where the typo “Formal-Table” produced an error and was retyped 39

Figure 34 – Get-Service Sysmon64 fails while WazuhSvc is Running (photo of screen, 5 Nov) 39

Figure 35 – Splunk query containing “CommandLLine” returns only the two encoded events 40

Figure 36 – Splunk refuses to run a search: minimum free disk space (5000 MB) reached (photo, 6 Nov 10:05) 40
Figure 37 – df -h /: 49 GB, 43 GB used, 4.0 GB available, 92% (photo, 6 Nov 10:10) 40
Figure 38 – After cleanup: 37 GB used, 9.9 GB available, 79%; /var/ossec is 14 GB (photo, 6 Nov) 41
Figure 39 – A 4104 search with a field filter returns “No results found” (photo, 6 Nov 10:22) 41
Figure 40 – Splunk table with an empty ProcessID column (photo, 6 Nov 10:27) 42

List of Tables

Table 1 – Abbreviations used in this report 8
Table 2 – Headline results (each is evidenced; see the figure references) 9
Table 3 – Day 6 requirements mapped to success criteria and outcome 11
Table 4 – The six stages and what can go wrong at each 13
Table 5 – Machines and tools actually used in Day 6 15
Table 6 – What the Step 1 commands mean 17
Table 7 – What the test commands mean 21
Table 8 – Facts the brief asks us to identify, for the 3:07 PM test (from Figs. 12–15) 22
Table 9 – What existed, what changed, what is only recommended 25
Table 10 – Explanation of each line 25
Table 11 – Search queries that worked 26
Table 12 – Every test, its expected and actual result, and verification status 27
Table 13 – Key events (local time, UTC+1) 27
Table 14 – Answers, each tied to evidence 28
Table 15 – Issues, causes and fixes 38
Table 16 – Findings, evidence and confidence 43
Table 17 – Implemented versus recommended 44
Table 18 – Skills mapped to evidence 46
Table 19 – If you do not see the expected result 48
Table 20 – Figure-to-original evidence register 52
Table 21 – Plain-language glossary 56

Abbreviations and Acronyms

Table 1 – Abbreviations used in this report

Term	Meaning (plain language)
SOC	Security Operations Centre: the team that watches for and responds to cyber attacks.
SIEM	Security Information and Event Management: software that collects logs from many computers and raises alerts (Wazuh and Splunk here).
EDR / agent	A small program on a computer that reports what happens on it. Here: the Wazuh agent.
Sysmon	System Monitor: a free Microsoft tool that records detailed process, network and file activity.
Event ID / EventCode	A number that says what kind of log entry this is (for example 4104 = PowerShell script text; Sysmon 1 = a program started).
4104	Windows PowerShell “script block logging” event: the text of code PowerShell ran.
MITRE ATT&CK	A public catalogue of attacker techniques. Each has an ID such as T1059.001.
GUID	A long unique identifier. Sysmon gives every process a ProcessGuid so it can be tracked precisely.
PID	Process ID: the number Windows gives a running program.
SPL / DQL	Search languages: Splunk Processing Language and the Wazuh dashboard query language.
False positive	An alert that fires on harmless activity.
IOC / IOA	Indicator of Compromise / Attack: a clue that something bad has happened or is happening.
VM	Virtual Machine: a computer simulated inside VirtualBox.
UTC / WAT	UTC is world time; Nigeria (WAT) is UTC+1. Screenshots show local time, Sysmon also stores UTC.

Chapter 1: Executive Summary

> **IN PLAIN ENGLISH**
> We switched on detailed recording of PowerShell on a Windows 10 test computer, sent those records to Wazuh and Splunk, ran harmless but suspicious-looking commands, and confirmed the monitoring tools noticed. We also wrote one detection rule of our own, and it worked.

1.1 Purpose and problem

Day 6 of the 14-day Wazuh SOC Analyst Home Lab asks the learner to *"develop visibility into PowerShell execution and investigate suspicious command activity."* PowerShell is a normal Windows administration tool that is also one of the most common tools used by attackers (MITRE ATT&CK technique T1059.001). Without extra logging, defenders cannot see what was run or how it was started.

1.2 Method in six lines

1. Enable PowerShell script-block and module logging on the Windows 10 VM.
2. Confirm Sysmon (process-level recording) is running.
3. Configure the Wazuh agent to collect both the PowerShell and Sysmon logs, and confirm they arrive at the Ubuntu-hosted Wazuh manager.
4. Run three harmless test commands: an encoded command with execution-policy bypass, and a hidden-window command.
5. Investigate the evidence in Sysmon, Wazuh and Splunk; write a custom Wazuh rule; map to MITRE ATT&CK.
6. Document false-positive risks and answer the six analyst questions.

1.3 Verified outcomes

Table 2 – Headline results (each is evidenced; see the figure references)

Area	Result	Evidence
PowerShell logging	Script-block and module logging both set to 1; 4104 events visible locally	Figs. 4–6
Collection	Wazuh agent running; Sysmon running; both log channels in ossec.conf; PowerShell event found in manager archive	Figs. 7–11
Sysmon context	User, parent process, command line and ProcessGuid captured for every test run	Figs. 12–15
Built-in alerts	Rule 92057 (level 12, encoded command) fired twice; rule 91815 (level 4, process discovery) fired	Figs. 16–23
Custom rule	Rule 100600 (level 10, hidden window) fired once on 6 Nov 08:44:21; MITRE T1059.001 and T1564.003	Figs. 24–26
Splunk cross-check	5 matching Sysmon events; PowerShell 4104 script text matches Wazuh by record number 1889 and script-block ID	Figs. 27–32
Problems solved	Missing registry value, wrong service name, Splunk typos, disk full (4.0 GB → 9.9 GB free)	Chapter 10

1.4 Not proven

- Splunk could not show the process ID (the classic Windows event format has no such field); correlation used record number and script-block ID instead.

- False positives were **not** measured: no harmless hidden-window command was run to see whether rule 100600 would also fire.
- The custom rule file and its syntax-check output were not captured in a screenshot; the alert itself proves the rule loaded and fired.

Chapter 2: Introduction and Objectives

2.1 Background and motivation

A junior SOC analyst spends much of the day asking: *what happened, how do I know, and what should I do?* This lab practises that loop on one of the most common attack tools. The brief for Day 6 (Figure 1) is the source of the requirements and the design reference for this report: navy title, blue sub-headings, bullet lists and a footer line reading “Wazuh SOC Analyst Home Lab — 14-Day Intensive Program”.

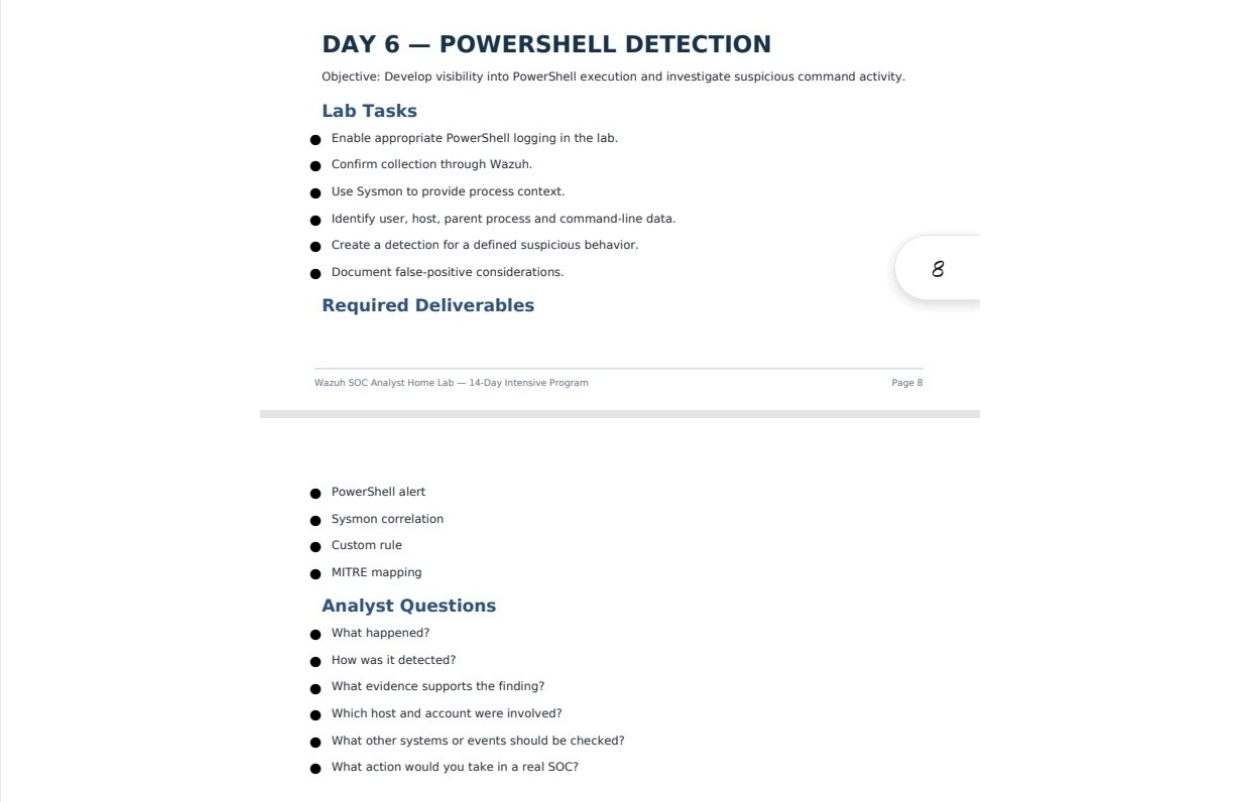


Figure 1 – The Day 6 lab brief (source of the tasks, deliverables and analyst questions; also the layout reference for this report)

2.2 Scope

In scope: one Windows 10 VM (monitored), one Ubuntu VM (Wazuh manager, dashboard and Splunk). **Out of scope:** the Kali attacker VM and the Windows Server domain controller (deliberately left out to keep the lab small), any real malware, and any production system.

2.3 Objectives and success criteria

Table 3 – Day 6 requirements mapped to success criteria and outcome

#	Requirement from the brief	Success criterion	Outcome
1	Enable appropriate PowerShell logging	Registry values = 1; 4104 events appear	Met (Figs. 4–6)
2	Confirm collection through Wazuh	Event found on the manager	Met (Fig. 11)
3	Use Sysmon for process context	Event 1 shows process details	Met (Figs. 12–15)
4	Identify user, host, parent process, command line	All four recorded for each test	Met (Step 6)
5	Create a detection for a defined suspicious behaviour	Custom rule fires on the behaviour	Met, one firing (Figs. 24–26)

#	Requirement from the brief	Success criterion	Outcome
6	Document false-positive considerations	Written analysis	Documented; not measured (Ch. 11, 16)
D1–4	Deliverables: PowerShell alert, Sysmon correlation, custom rule, MITRE mapping	Evidence for each	Met

Chapter 3: Technical Background

> **IN PLAIN ENGLISH**
> Read this chapter like a glossary with stories. You do not need to memorise it; the later chapters refer back to it.

3.1 The detection pipeline

Beginners often mix up six different ideas. Keeping them separate makes troubleshooting much easier, because a failure is always at *one* stage.

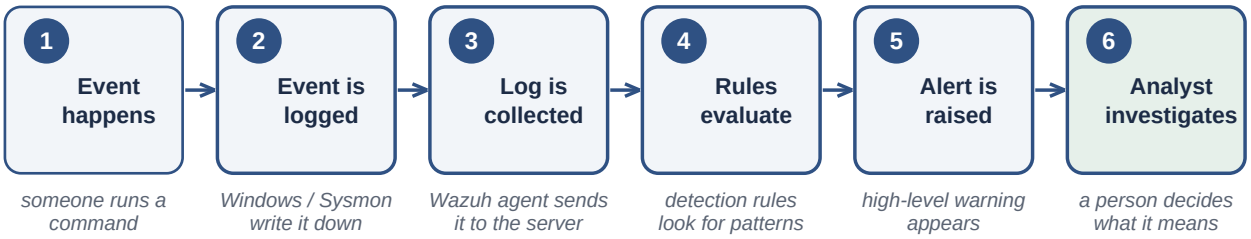


Figure 2 – The six stages from event to investigation. A problem at any stage looks like "no alert".

Table 4 – The six stages and what can go wrong at each

Stage	Meaning	Example in this lab	Typical failure
1 Event	Something happens on the computer	PowerShell is started with -EncodedCommand	None (it happened)
2 Logged	Windows or Sysmon writes it to a log	Event 4104 and Sysmon Event 1	Logging not enabled (Step 1)
3 Collected	Agent reads the log and sends it	Wazuh agent reads the two channels	Channel missing from ossec.conf
4 Evaluated	Rules compare the event to patterns	Rules 92057, 91815, 100600	Rule not loaded or syntax error
5 Alert	A matching event becomes an alert	Dashboard row with rule ID and level	Level too low; disk full
6 Investigate	A person reads the evidence	Analyst questions in Chapter 8	Reading the wrong time or host

3.2 Key concepts

PowerShell and why attackers use it

PowerShell is a command-line and scripting tool built into Windows. It can run programs, read files, change settings and talk to the network. Because it is trusted and already present, attackers use it to avoid installing their own tools. MITRE ATT&CK lists this as **T1059.001 (Command and Scripting Interpreter: PowerShell)**.

Encoded commands (base64)

PowerShell can accept a command scrambled into base64 text with the flag `-EncodedCommand`. Base64 is not encryption; it simply turns text into a string of letters and digits. Admins sometimes use it legitimately, but attackers use it to hide intent from a casual glance. In this lab the test command was

`whoami; hostname`, which becomes:

```
dwBoAG8AYQBtAGkAOwAgAGgAbwBzAHQAbgBhAG0AZQA=
```

Anyone can reverse it; we decoded the string above and it returns exactly *whoami; hostname*. The "UTF-16" detail matters: PowerShell encodes text as UTF-16, which is why the string looks longer than expected.

Execution-policy bypass and hidden windows

`-ExecutionPolicy Bypass` tells PowerShell to ignore its script-running safety setting. `-WindowStyle Hidden` runs PowerShell without showing a window. Neither is malicious by itself; together with encoding they are a common attacker pattern, which is why they make useful detection ideas.

PowerShell logging: Event 4104 and 4103

Script-block logging records the text of code PowerShell executes as **Event 4104**, even if it was typed rather than saved as a file. **Module logging** records pipeline details as **Event 4103**. Both live in the Windows log channel *Microsoft-Windows-PowerShell/Operational*.

Sysmon

Sysmon is a free Microsoft tool that records detailed system activity. **Sysmon Event ID 1** means "a process was created" and records the program path, command line, user, the **parent process** (what started it), file hashes and a unique **ProcessGuid**. It answers "who started what, and from where?".

Wazuh, rules and levels

Wazuh has an **agent** (on the monitored computer) and a **manager** (the server). The manager decodes each event, compares it to **rules** and creates an **alert** with a rule ID and a **level** (0–15; higher is more serious). Rules shipped with Wazuh have IDs below 100000; your own rules start at 100000, which is why ours is **100600**.

Splunk

Splunk is a second SIEM. Events arrive in **indexes** (here *sysmon*, *windows* and *main*) and are searched with SPL. Using two SIEMs lets us cross-check that the evidence is real and not an artefact of one tool.

MITRE ATT&CK

ATT&CK gives every attacker behaviour a name and an ID so that analysts around the world speak the same language. Wazuh rules can carry these IDs; they then appear in the alert (Chapter 9).

Chapter 4: Lab Environment and Architecture

Table 5 – Machines and tools actually used in Day 6

Component	Details (from the evidence)
Windows 10 VM	Host name DESKTOP-T6T1N2V.HopeLabs.Local; IP 192.168.56.109; Wazuh agent ID 006; Windows build 10.0.19041; user HOPELABS\Administrator for the tests
Sysmon	Installed as service “Sysmon”; writes to Microsoft-Windows-Sysmon/Operational; schema version 5; Event ID 1 enabled
Ubuntu VM	Host name Ubuntu; IP 192.168.56.101; root disk /dev/sda2 (49 GB); Wazuh manager and dashboard; Splunk Enterprise 9.4.2
Wazuh	Alert index wazuh-alerts-4.x; manager name Ubuntu; archive logging (logall and logall_json) set to yes
Splunk indexes	sysmon (Sysmon events), windows (PowerShell, System, Application, Defender), main (Security)
Network	VirtualBox host-only network 192.168.56.0/24 between the VMs
Not used today	Kali Linux attacker VM; Windows Server 2019 domain controller

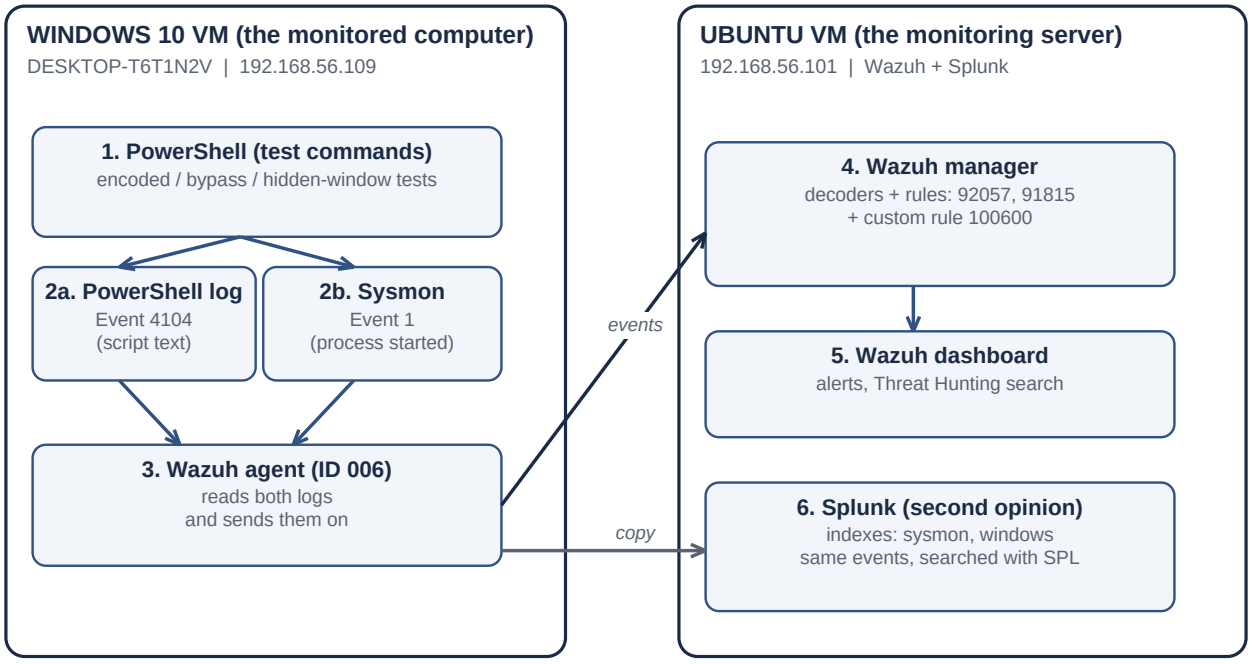


Figure 3 – Laboratory architecture and data flow (drawn from the IP addresses, agent ID and index names seen in the evidence)

4.1 How the parts talk to each other

- PowerShell runs on Windows 10; Windows writes script text to the PowerShell log (2a) and Sysmon records each process start (2b).
- The Wazuh agent reads both logs and sends the events to the Wazuh manager on Ubuntu (arrow 3→4).
- The manager applies rules and stores alerts, which the dashboard displays.
- A separate copy of the events is forwarded to Splunk, which stores them in its own indexes (the “second opinion”).

Chapter 5: Prerequisites and Safety

5.1 What you need before starting

- A Windows 10 VM with the Wazuh agent installed and showing as *active* in the dashboard, and Sysmon installed with a configuration that logs process creation.
- An Ubuntu VM running the Wazuh manager and dashboard (Splunk is optional but used here).
- Administrator rights on Windows; sudo rights on Ubuntu.
- The two VMs able to reach each other (here a host-only network).
- At least **10 GB free** on the Ubuntu root disk (see Chapter 10: Splunk stops searching below 5 GB free).

5.2 Safety rules

> **CAUTION**
> Only run these tests inside isolated lab VMs. The commands are harmless (they display the user name, host name and process list), but the *technique* is what attackers use. Never run an encoded command you did not write yourself, and never practise on a work or personal computer.

- **Take a VM snapshot first** (VirtualBox: Machine → Take Snapshot) so you can roll back.
- Keep the lab host-only; do not bridge it to a real network.
- Change registry and configuration files carefully and keep a copy of `ossec.conf` before editing.
- Remember that logging every event takes disk space (Chapter 10); do not leave archive logging on forever.

5.3 Restoring if something goes wrong

Revert the snapshot, or use the rollback steps in Section 15.9 (delete the three registry keys, remove the PowerShell block from `ossec.conf`, remove rule 100600, restart the services).

Chapter 6: Step-by-Step Implementation

> **IN PLAIN ENGLISH**
> Nine steps. Each step says *where* to do it, *why*, what you *expect* to see, and what we actually saw. Screenshots that prove the detection results are in Chapter 9.

Step 1 – Turn on PowerShell logging

Where	Windows 10 VM, PowerShell opened as Administrator (3 Nov 2026, about 12:49–12:57 PM)
Why	Windows does not record PowerShell script text by default. These registry settings turn on Script Block Logging (Event 4104) and Module Logging (Event 4103).
Expected	Each Get-ItemProperty shows the value 1; later a 4104 event appears for each command typed.
Actually observed	Both values showed 1. A first attempt created the ModuleNames key but the value line was missing from the screenshots, so it was set again (Figure 5).

Windows 10, elevated PowerShell

```
New-Item "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Force
Set-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" EnableScriptBlockLogging
New-Item "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging" -Force
Set-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging" EnableModuleLogging 1
New-Item "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging\ModuleNames" -Force
Set-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging\ModuleNames" "*" "*"
```

Table 6 – What the Step 1 commands mean

Part	Meaning
New-Item ... -Force	Creates the registry key (a folder of settings); -Force avoids an error if it exists
Set-ItemProperty ... 1	Sets a value inside the key; 1 means “on”
HKLM:\SOFTWARE\Policies\...	The Windows policy location read by PowerShell
ModuleNames “*” “*”	Log every PowerShell module (the asterisk is a wildcard)



Figure 4 – The registry keys being created (Windows 10, 3 Nov 2026, 12:57 PM)

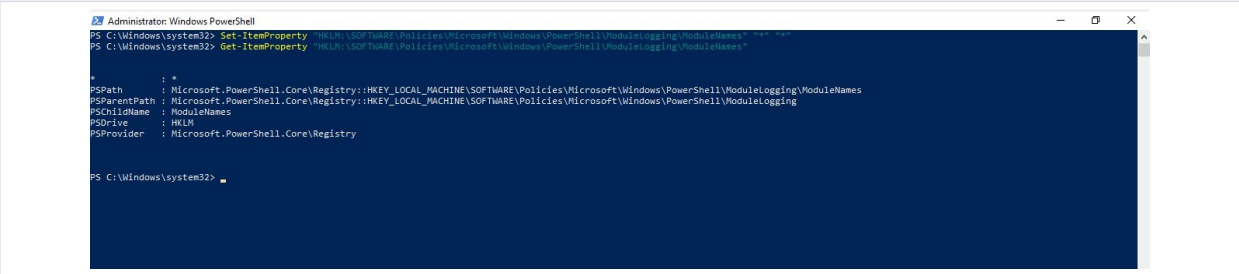


Figure 5 – ModuleNames set to "" and read back, completing module logging*

Verify logging works by running a harmless command and reading the log back:

Windows 10, elevated PowerShell

```
Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"
Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging"
Write-Output "logging test"
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" -MaxEvents 5 |
  Format-Table TimeCreated, Id, Message -Wrap
```

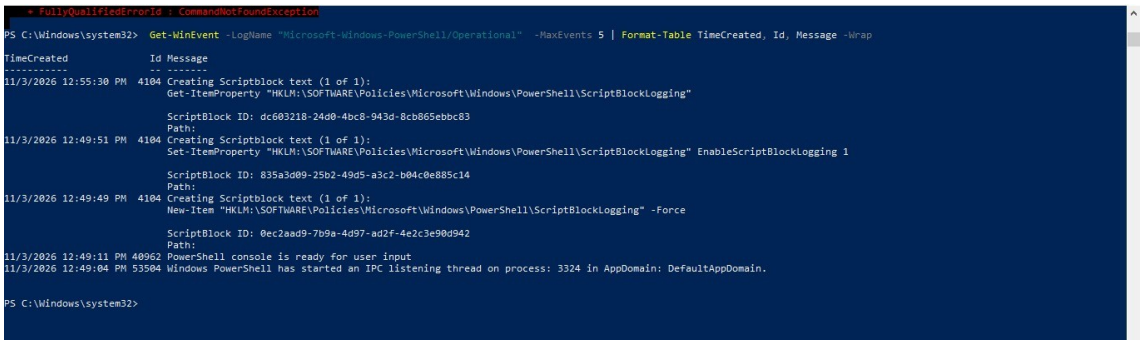


Figure 6 – Event ID 4104 entries showing the commands just typed (12:49–12:55 PM)

> **VERIFIED RESULT**
> PowerShell script-block logging is working: 4104 events captured the New-Item, Set-ItemProperty and Get-ItemProperty commands typed in the session.

Step 2 – Confirm the recorder (Sysmon) and agent (Wazuh) are running

Where	Windows 10 VM, elevated PowerShell (5 Nov 2026)
Why	If the agent or Sysmon is stopped, nothing reaches Wazuh and the rest of the lab fails silently.
Expected	Both services show Status = Running.
Actually observed	The Wazuh agent (WazuhSvc) was Running. Sysmon was not found under the name Sysmon64 but was found as service "Sysmon", Running (see Ch. 10, Issue 2).

Windows 10, elevated PowerShell

```
Get-Service Sysmon64, WazuhSvc
Get-Service | Where-Object { $_.Name -like "*sysmon*" -or $_.DisplayName -like "*sysmon*" }
```

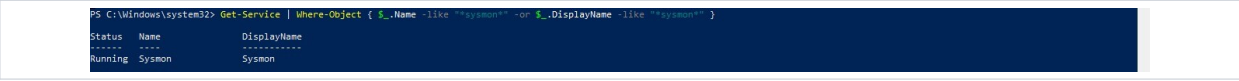


Figure 7 – Sysmon found running under the service name "Sysmon"

Step 3 – Tell the Wazuh agent to collect the two log channels

Where	Windows 10 VM; file C:\Program Files (x86)\ossec-agent\ossec.conf, edited in Notepad as Administrator (5 Nov, 2:48–2:59 PM)
Why	The agent only reads logs it is told about. Each block names one Windows log channel.
Expected	ossec.conf contains one block for Sysmon and one for PowerShell; the agent restarts and stays Running.
Actually observed	The Sysmon block already existed; the PowerShell block was added. Select-String found both at lines 50 and 65, and the agent restarted cleanly.

ossec.conf (add only the block that is missing)

```
<localfile>
  <location>Microsoft-Windows-PowerShell/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Windows 10, elevated PowerShell

```
notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"
Restart-Service WazuhSvc
Get-Service WazuhSvc
Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "PowerShell/Operational","Sysmon/Opera
```

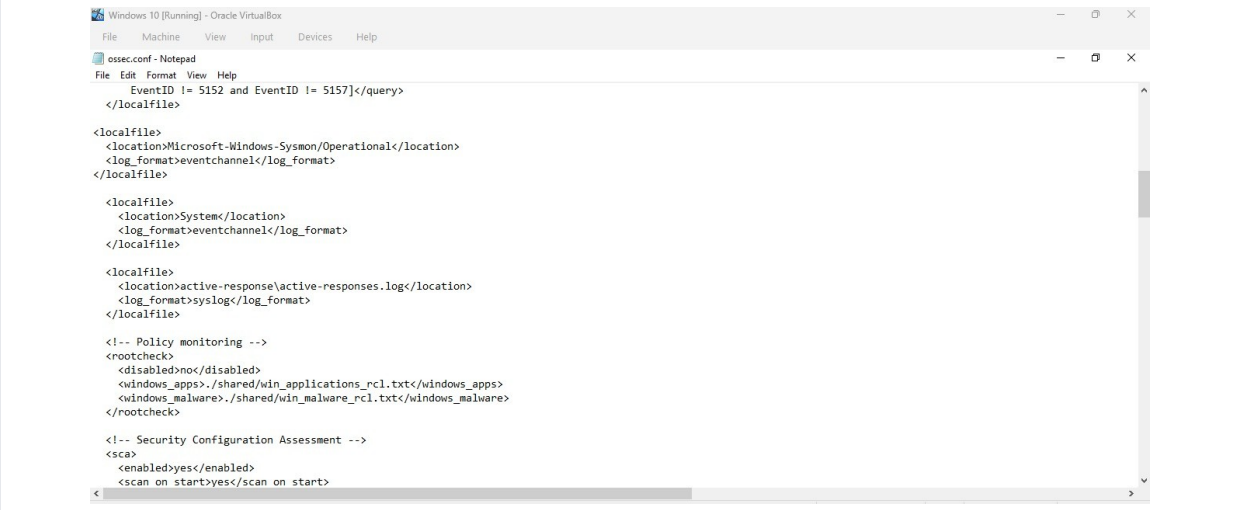


Figure 8 – ossec.conf in Notepad: the existing Sysmon block (5 Nov 2026, 2:48 PM)



Figure 9 – Agent restarted and reported Running (2:55 PM)

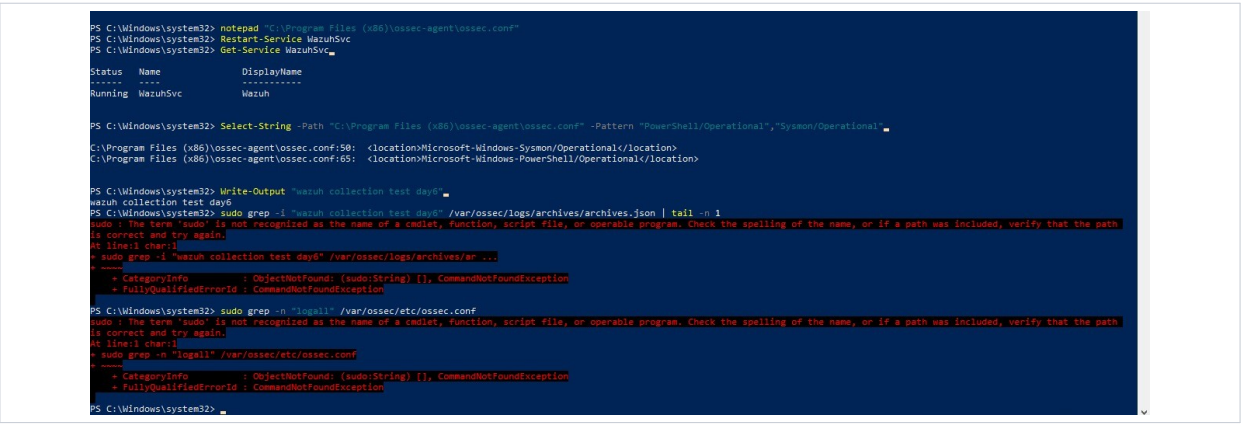


Figure 10 – Select-String shows both channels at lines 50 and 65 (2:59 PM). The two red sudo errors are explained in Chapter 10, Issue 3.

Step 4 – Prove the manager receives the events

Where	Ubuntu VM, terminal (5 Nov, about 3:03 PM)
Why	Collection on the agent does not prove delivery. The manager archive records every event received, even when no rule alerts on it.
Expected	A JSON line containing the test text appears in archives.json.
Actually observed	A 4104 event from agent 006 containing the test phrase was found. (It was the script text of a command typed in the wrong window, which is itself a nice demonstration of script-block logging.)

Ubuntu VM terminal (requires sudo)

```
sudo grep -i "wazuh collection test day6" /var/ossec/logs/archives/archives.json | tail -n 1
sudo grep -n "logall" /var/ossec/etc/ossec.conf
```

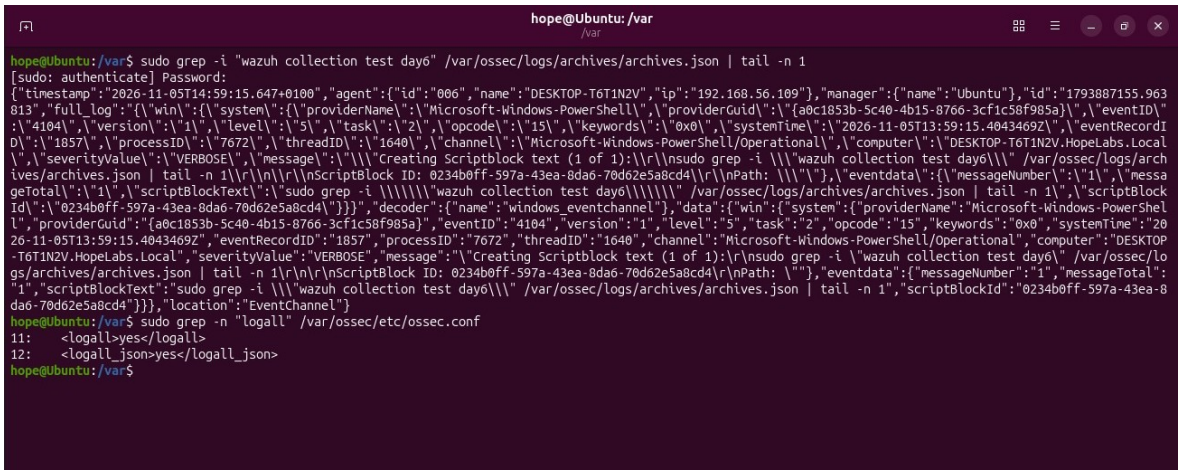


Figure 11 – Archive entry for agent DESKTOP-T6T1N2V, channel PowerShell/Operational, Event ID 4104; logall and logall_json set to yes

Step 5 – Generate the test activity

Where	Windows 10 VM, elevated PowerShell (5 Nov, 3:05 PM and 3:07 PM)
Why	We need something suspicious-looking but harmless to detect. The commands only print the user name and host name, or list processes.
Expected	The first command prints the user and host; the second runs silently.
Actually observed	Run twice (3:05 PM and 3:07 PM). A screenshot of the commands being typed was not captured; the activity is proven by the logs in Steps 6–9. The hidden-window command was run again on 6 Nov at 08:44 to test the custom rule.

Windows 10, elevated PowerShell

```
$cmd = 'whoami; hostname'
$enc = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($cmd))
powershell.exe -NoProfile -ExecutionPolicy Bypass -EncodedCommand $enc
powershell.exe -NoProfile -WindowStyle Hidden -Command "Get-Process | Out-Null"
```

Table 7 – What the test commands mean

Part	Meaning
$cmd = 'whoami; hostname'	The harmless text we want to run
ToBase64String(...Unicode.GetBytes($cmd))	Scrambles it into base64 (the encoded form attackers use)
-NoProfile	Skip the user's start-up script
-ExecutionPolicy Bypass	Ignore the script-running restriction
-EncodedCommand $enc	Run the scrambled text
-WindowStyle Hidden	Do not show a window
Get-Process | Out-Null	List running programs, discard the output

Step 6 – Read the Sysmon process context locally

Where	Windows 10 VM, elevated PowerShell (5 Nov, about 3:08–3:19 PM)
Why	Sysmon Event 1 records user, parent process, command line and ProcessGuid, which are the facts the brief asks us to identify.
Expected	Event 1 entries for powershell.exe with the test command lines.
Actually observed	Found for both runs. The default view of the last five events was swamped by background activity (Edge updates), so the output was filtered by keyword.

Windows 10, elevated PowerShell

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1} -MaxEvents 200 |
  Where-Object { $_.Message -match 'EncodedCommand|WindowStyle Hidden' } |
  Format-List TimeCreated, Message
```

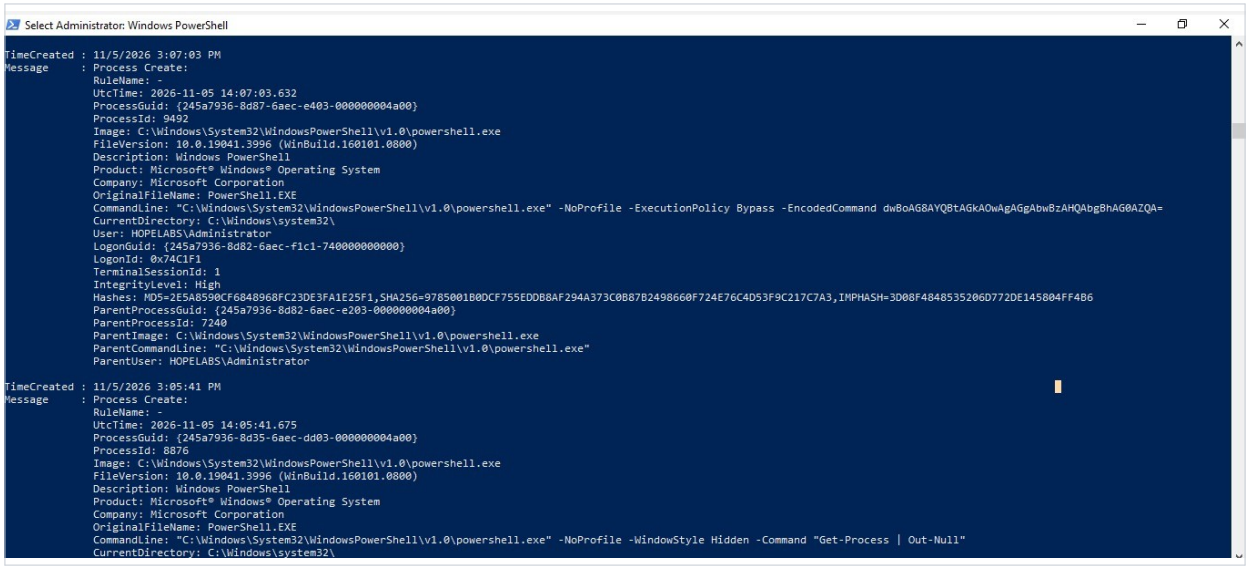


Figure 12 – Sysmon Event 1: encoded PowerShell (PID 9492) started at 3:07:03 PM by parent PID 7240

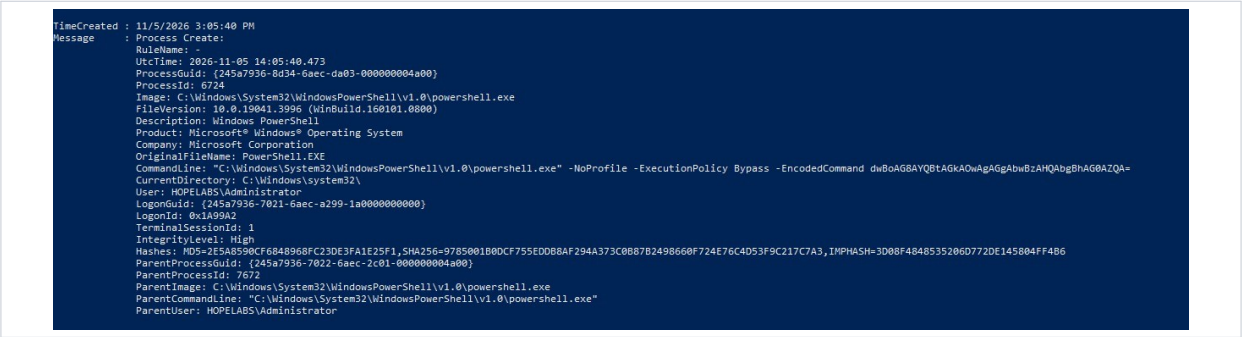


Figure 13 – Sysmon Event 1: the earlier encoded run (PID 6724) at 3:05:40 PM, parent PID 7672

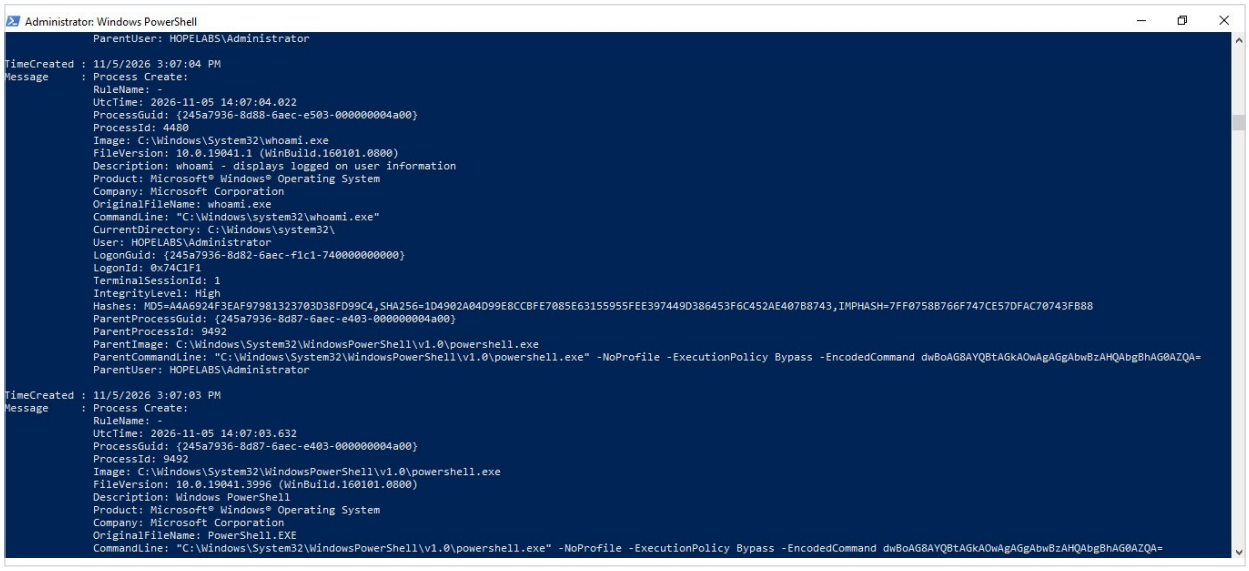


Figure 14 – Child processes whoami.exe and HOSTNAME.EXE; their ParentCommandLine is the encoded command

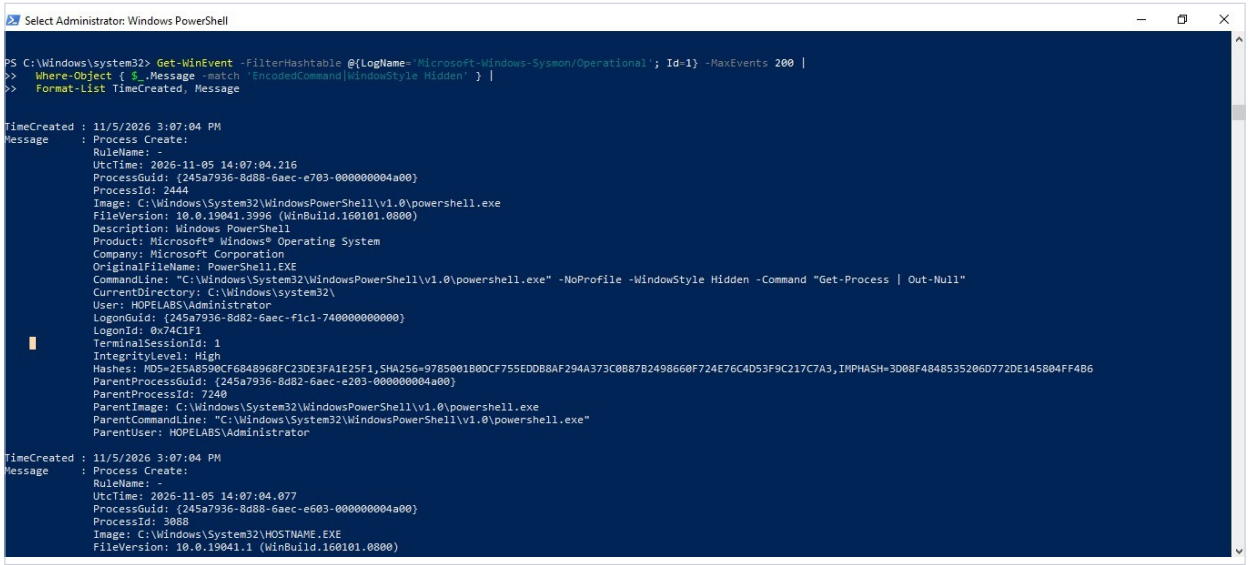


Figure 15 – Sysmon Event 1: hidden-window PowerShell (PID 2444) at 3:07:04 PM, parent PID 7240

Table 8 – Facts the brief asks us to identify, for the 3:07 PM test (from Figs. 12–15)

Question	Encoded command	Hidden-window command
Host	DESKTOP-T6T1N2V	DESKTOP-T6T1N2V
User	HOPELABS\Administrator (High integrity)	HOPELABS\Administrator (High integrity)
Process	powershell.exe, PID 9492	powershell.exe, PID 2444
Parent process	powershell.exe, PID 7240	powershell.exe, PID 7240

Question	Encoded command	Hidden-window command
Command line	-NoProfile -ExecutionPolicy Bypass -EncodedCommand dwBoAG8A...	-NoProfile -WindowStyle Hidden -Command "Get-Process | Out-Null"
ProcessGuid	{245a7936-8d87-6aec-e403-000000004a00}	{245a7936-8d88-6aec-e703-000000004a00}
Children	whoami.exe (PID 4480), HOSTNAME.EXE (PID 3088)	none shown

VERIFIED RESULT
Both test runs share the same parent (PID 7240): one parent PowerShell session launched both the encoded and the hidden-window processes. Decoding the base64 payload gives “whoami; hostname”, and the child processes whoami.exe and HOSTNAME.EXE confirm it actually ran.

Step 7 – Find the built-in Wazuh alerts

Where	Ubuntu VM, Wazuh dashboard → Threat Hunting → Events, agent DESKTOP-T6T1N2V (006), last 24 hours (6 Nov, 08:16–08:29)
Why	This shows which rules already catch the behaviour without any custom work.
Expected	At least one alert referencing the encoded command.
Actually observed	Rule 92057 (level 12) fired for both encoded runs; rule 91815 (level 4) fired on the process-discovery command. Details in Chapter 9.

Wazuh dashboard query bar (DQL)

```
data.win.eventdata.commandLine: *EncodedCommand*
data.win.system.eventID: 4104 AND data.win.eventdata.scriptBlockText: *Bypass*
```

Step 8 – Write and test the custom rule

Where	Ubuntu VM, file /var/ossec/etc/rules/local_rules.xml (6 Nov, before 08:44)
Why	No built-in alert specifically named the hidden-window behaviour, so we wrote one (Chapter 7 explains every line).
Expected	After restarting the manager and re-running the hidden-window command, an alert with rule ID 100600 appears.
Actually observed	Rule 100600 fired once at 08:44:21 on 6 Nov (level 10) with MITRE T1059.001 and T1564.003 (Chapter 9, Figs. 24–26).

Ubuntu VM terminal

```
sudo grep -rn "100600" /var/ossec/etc/rules/        # empty output = ID is free
sudo nano /var/ossec/etc/rules/local_rules.xml      # add the rule from Chapter 7
sudo /var/ossec/bin/wazuh-analysisd -t              # syntax check (should print no errors)
sudo systemctl restart wazuh-manager
```

EVIDENCE GAP
Screenshots of the rule file and of the syntax-check output were not captured. That the rule loaded and fired is proven by the alert itself (Figure 24).

Step 9 – Cross-check in Splunk

Where	Ubuntu VM, Splunk Search & Reporting, last 24 hours (6 Nov, 09:21–10:37)
Why	A second SIEM that agrees with Wazuh is stronger evidence than one tool alone.
Expected	The same five PowerShell Sysmon events and the same 4104 script text.

Actually observed	Five Sysmon events and the 4104 events were found after fixing typos and a full-disk problem (Chapter 10). Details in Chapter 9.

Chapter 7: Configuration and Detection Logic

7.1 Original configuration versus changes

Table 9 – What existed, what changed, what is only recommended

Item	Before	Change made	Recommended later
PowerShell logging	Not enabled	Script-block + module logging enabled (registry)	Add transcription logging
ossec.conf (agent)	Sysmon channel present	PowerShell/Operational channel added	None
Wazuh archives	logall / logall_json = yes already	None	Turn off or rotate after the lab
local_rules.xml	No rule 100600	Rule 100600 added	Tune with allow-list; add combined rule
Ubuntu disk	4.0 GB free (92% used)	Journals and apt cache cleaned: 9.9 GB free	Expand virtual disk; log rotation

7.2 The custom rule, line by line

/var/ossec/etc/rules/local_rules.xml (as supplied in the lab; the alert in Fig. 24 confirms it loaded)

```
<group name="powershell,custom,">
  <rule id="100600" level="10">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.image" type="pcre2">(?i)\\powershell\.exe$</field>
    <field name="win.eventdata.commandLine" type="pcre2">(?i)-w(indowstyle)?\s+hidden</field>
    <description>Suspicious PowerShell: hidden window execution</description>
    <mitre>
      <id>T1059.001</id>
      <id>T1564.003</id>
    </mitre>
  </rule>
</group>
```

Table 10 – Explanation of each line

Line	Plain meaning
group name="powershell,custom,"	Labels the alert. The alert shows these groups (Fig. 26).
rule id="100600" level="10"	Our own ID (custom rules start at 100000). Level 10 = high, but below the built-in level 12 for encoded commands.
if_sid 61603	Only look at events already classified by Wazuh as Sysmon Event 1 (a process was created).
field ...image ... powershell.exe	The program must be powershell.exe. (?i) = ignore capital letters; $ = the path must end there.
field ...commandLine ... hidden	The command line must contain -w hidden or -windowstyle hidden. -w(indowstyle)? matches both short and long forms.
description	The text shown on the alert (Fig. 24).
mitre T1059.001, T1564.003	PowerShell and Hidden Window techniques, shown on the alert (Fig. 26).

7.3 Why these conditions, and what they miss

- The rule is deliberately narrow: it fires on a known-suspicious flag, not on all PowerShell.

- It would **not** catch other ways to hide PowerShell (for example launching it from a script host or using `-w 1`, which PowerShell also accepts as a hidden style). That is a limitation (Chapter 16).
- It would **also fire** on legitimate hidden-window PowerShell (deployment tools, scheduled tasks). That is a false-positive risk (Chapter 11); we did not measure it.

7.4 Queries used

Table 11 – Search queries that worked

Tool	Query (shortened)	Purpose
Wazuh DQL	data.win.eventdata.commandLine: *EncodedCommand*	Find the encoded-command alerts
Wazuh DQL	rule.id: 100600	Find the custom-rule alert
Splunk	index=* host="DESKTOP-T6T1N2V" | stats count by index, sourcetype, source	See what data Splunk holds
Splunk	index=sysmon source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 Image="*powershell.exe" (CommandLine="*-EncodedCommand*" OR CommandLine="*-WindowStyle Hidden*")	The five suspicious PowerShell launches
Splunk	index="windows" source="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104 "*EncodedCommand*" | table _time host	Matching 4104 script-block events

Chapter 8: Testing and Results

8.1 Test results

Table 12 – Every test, its expected and actual result, and verification status

#	Test	Expected	Actual	Status
T1	Script-block logging enabled	Registry = 1	1	Verified
T2	Module logging enabled	Registry = 1; ModuleNames set	1; value set (second attempt)	Verified (4103 events not inspected)
T3	4104 events locally	Events for typed commands	Present	Verified
T4	Wazuh agent and Sysmon running	Both Running	Both Running (Sysmon named “Sysmon”)	Verified
T5	ossec.conf has both channels	Two matches	Lines 50 and 65	Verified
T6	Manager receives PowerShell events	Archive entry	Entry found	Verified
T7	Sysmon Event 1 context	User, parent, command line, GUID	All present	Verified
T8	Encoded command alert	Built-in alert	Rule 92057, level 12, ×2	Verified
T9	Process-discovery alert	Alert on Get-Process	Rule 91815, level 4	Verified
T10	Hidden window: built-in alert	Unknown	No built-in hidden-window alert seen in evidence	Inconclusive
T11	Custom rule 100600 fires	Alert	1 alert, 6 Nov 08:44:21, level 10	Verified (one firing)
T12	Rule false positives	Measure benign hidden-window use	Not run	Not tested
T13	Splunk shows same events	5 Sysmon + 4104 events	5 Sysmon events; 4104 events found	Verified
T14	Splunk process-ID correlation	PID in event	No PID field in the event	Not possible; used record number + script-block ID
T15	Rule syntax check output	No errors	Not captured	Evidence gap

8.2 Timeline of the lab

Table 13 – Key events (local time, UTC+1)

When	Event
3 Nov, 12:49–12:57 PM	PowerShell script-block and module logging configured and checked (Step 1)
5 Nov, 2:48–2:59 PM	ossec.conf edited; agent restarted; both channels confirmed (Step 3)
5 Nov, ~3:03 PM	Manager archive shows the PowerShell event (Step 4)
5 Nov, 3:05:40 PM	Test run 1: encoded command (PID 6724) and hidden-window command
5 Nov, 3:07:03 PM	Test run 2: encoded command (PID 9492) and hidden-window command (PID 2444)
5 Nov, 3:08–3:19 PM	Sysmon events read and filtered locally (Step 6)
5 Nov, 3:13–3:14 PM	Analyst investigation commands logged as extra 4104 events (see Section 11.2)
6 Nov, 08:16–08:29	Wazuh alerts 92057 and 91815 reviewed (Step 7)

When	Event
6 Nov, 08:44:21	Custom rule 100600 fires (Step 8)
6 Nov, 09:21–10:37	Splunk checks; typo fixes; disk-full error at about 10:05 and recovery (Chapter 10)

8.3 The six analyst questions from the brief

Table 14 – Answers, each tied to evidence

Question	Answer
What happened?	On DESKTOP-T6T1N2V, an administrator-level PowerShell session (PID 7240) launched a base64-encoded command with execution-policy bypass (decoded: “whoami; hostname”) and a hidden-window process listing, twice on 5 Nov and once more on 6 Nov.
How was it detected?	Sysmon Event 1 and PowerShell Event 4104 were collected by Wazuh. Built-in rule 92057 (encoded command, level 12) and 91815 (process discovery, level 4) fired; custom rule 100600 (hidden window, level 10) fired. Splunk showed the same events.
What evidence supports it?	Command lines, parent-child chain (7240 → 9492 → whoami/HOSTNAME), ProcessGuids, 4104 script text, matching Wazuh and Splunk records (record 1889, script-block ID cbca6012-ab6e-4c16-990d-35b17d841dc8).
Which host and account?	DESKTOP-T6T1N2V (192.168.56.109); account HOPELABS\Administrator, High integrity.
What else should be checked?	The same command lines on other hosts; Sysmon Event 3 (network connections) from these processes; new scheduled tasks or Run keys; other logons by the account; files created or downloaded; the 4104 text of the decoded payload itself.
What would you do in a real SOC?	Decide whether it was authorised (ask the admin, check change tickets). If not: isolate the host, preserve the logs, decode and analyse the payload, reset the account’s credentials, hunt for the same command line across the fleet, and record everything in a ticket. Here it was our own test, so no action beyond documentation was needed.

Chapter 9: Evidence and Screenshot Analysis

> **IN PLAIN ENGLISH**
> This chapter is the proof. Each screenshot is followed by a small table: what it shows, what it demonstrates, and what it cannot prove.

9.1 Built-in alert: encoded PowerShell command (rule 92057)

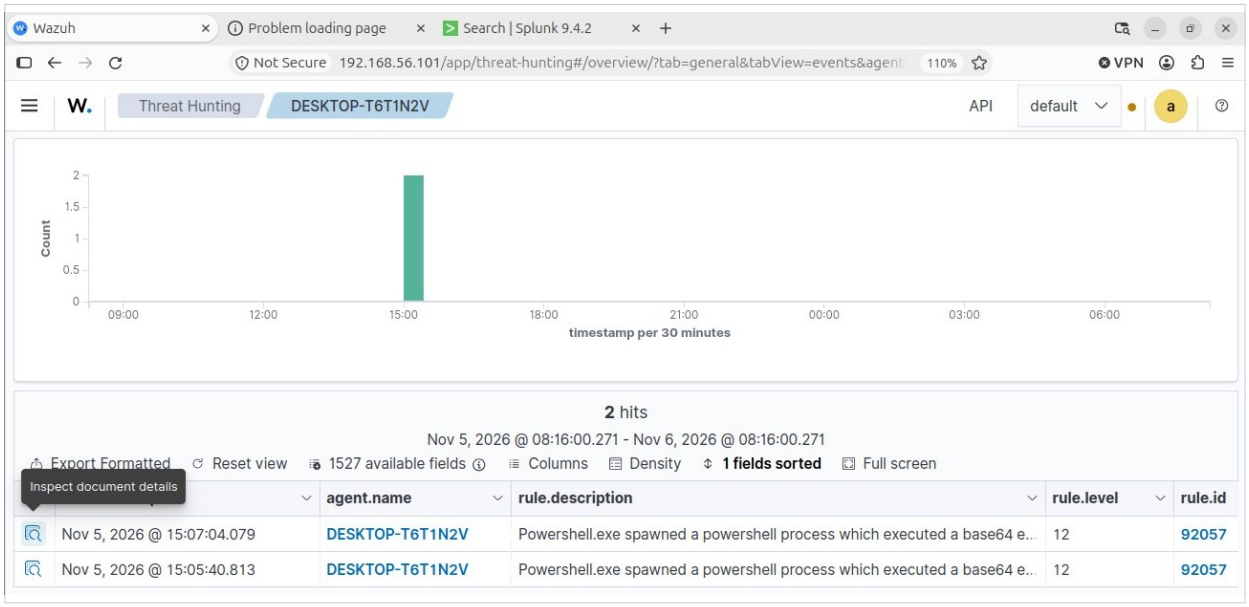


Figure 16 – Wazuh Threat Hunting results for the encoded-command query: two level-12 alerts (6 Nov 2026)

Shows	Two hits for agent DESKTOP-T6T1N2V at 15:07:04.079 and 15:05:40.813 on 5 Nov; description “Powershell.exe spawned a powershell process which executed a base64 encoded command”; level 12; rule 92057.
Demonstrates	The built-in ruleset detects the encoded command with no custom work, and both test runs were alerted.
Limitation	Only the alert row is visible; the underlying fields are in the next figures.

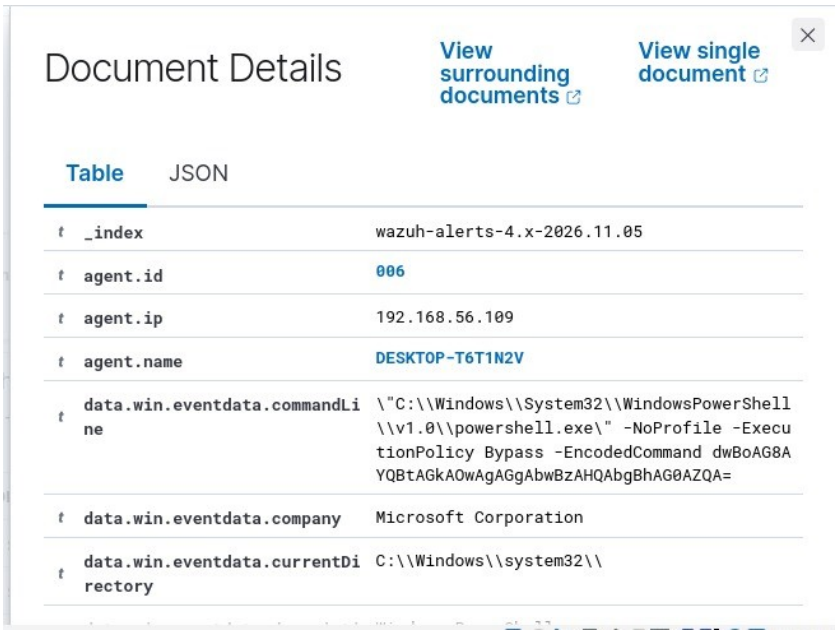


Figure 17 – Alert fields: the command line as seen by Wazuh

Shows	commandLine ends in “-NoProfile -ExecutionPolicy Bypass -EncodedCommand dwBoAG8A...”; index wazuh-alerts-4.x-2026.11.05; agent 006, IP 192.168.56.109.
Demonstrates	The alert carries the full command line, so the analyst can see the bypass and encoded flags without leaving the dashboard.
Limitation	The base64 text must still be decoded by the analyst (Section 3.2).

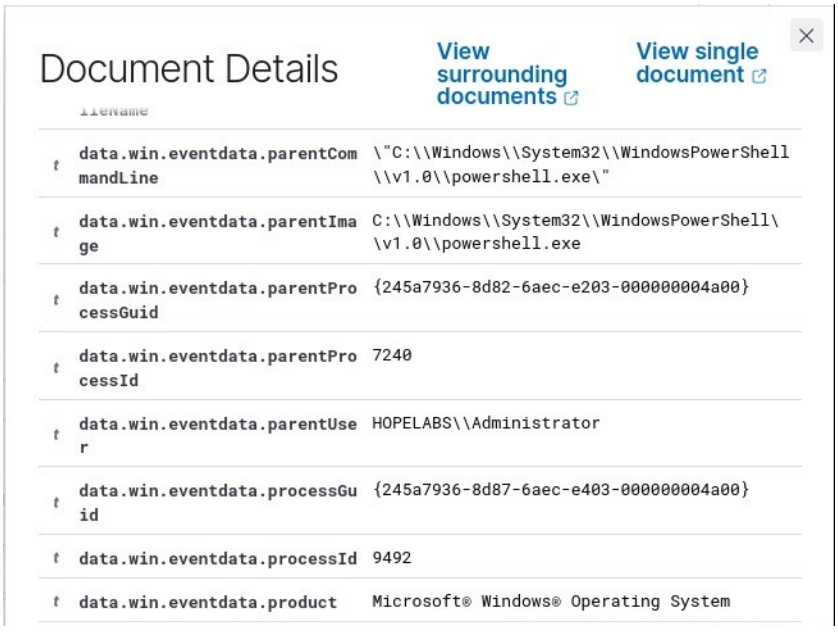


Figure 18 – Alert fields: parent process and process IDs

Shows	parentProcessId 7240; parentUser HOPELABS\Administrator; processId 9492; processGuid {245a7936-8d87-6aec-e403-000000004a00}.
Demonstrates	Wazuh preserves the Sysmon parent-child link, matching the local Sysmon evidence (Fig. 12).
Limitation	None for this claim.

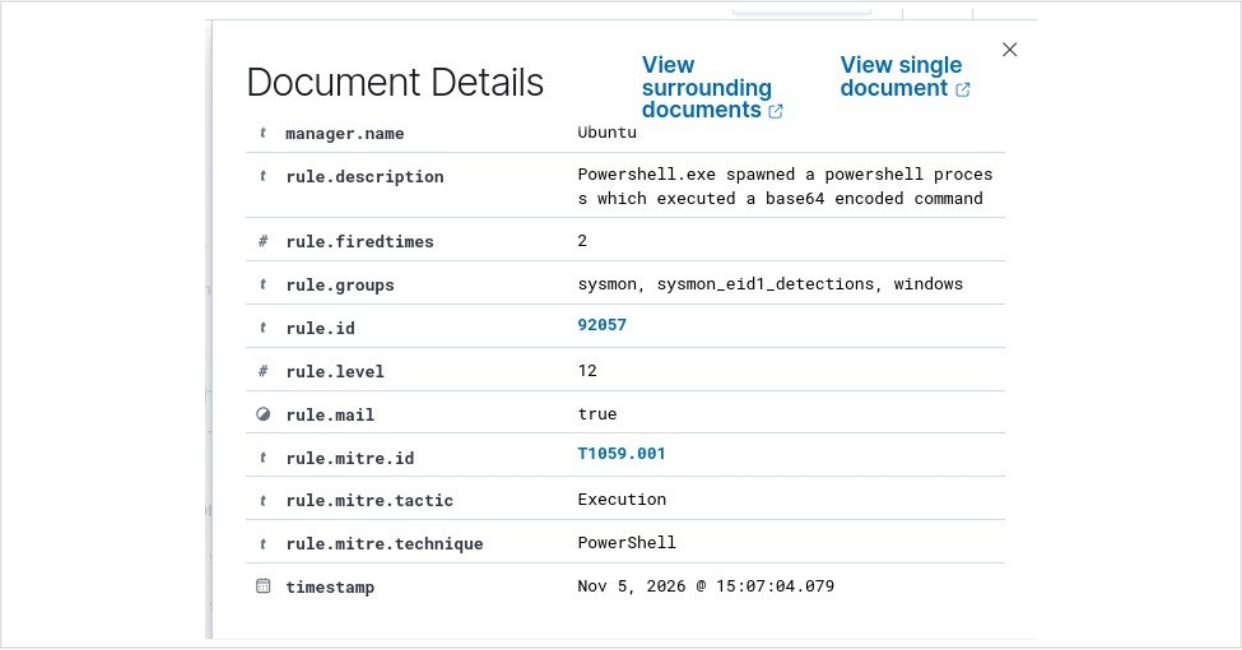


Figure 19 – Alert fields: rule metadata for 92057

Shows	rule.id 92057; level 12; groups sysmon, sysmon_eid1_detections, windows; mitre.id T1059.001 (Execution, PowerShell); mail true; firedtimes 2.
Demonstrates	The built-in alert maps to MITRE T1059.001.
Limitation	The rule only lists T1059.001; the “obfuscated command” idea (T1027) is the analyst’s mapping, not the rule’s.

9.2 PowerShell script block (4104) and rule 91815

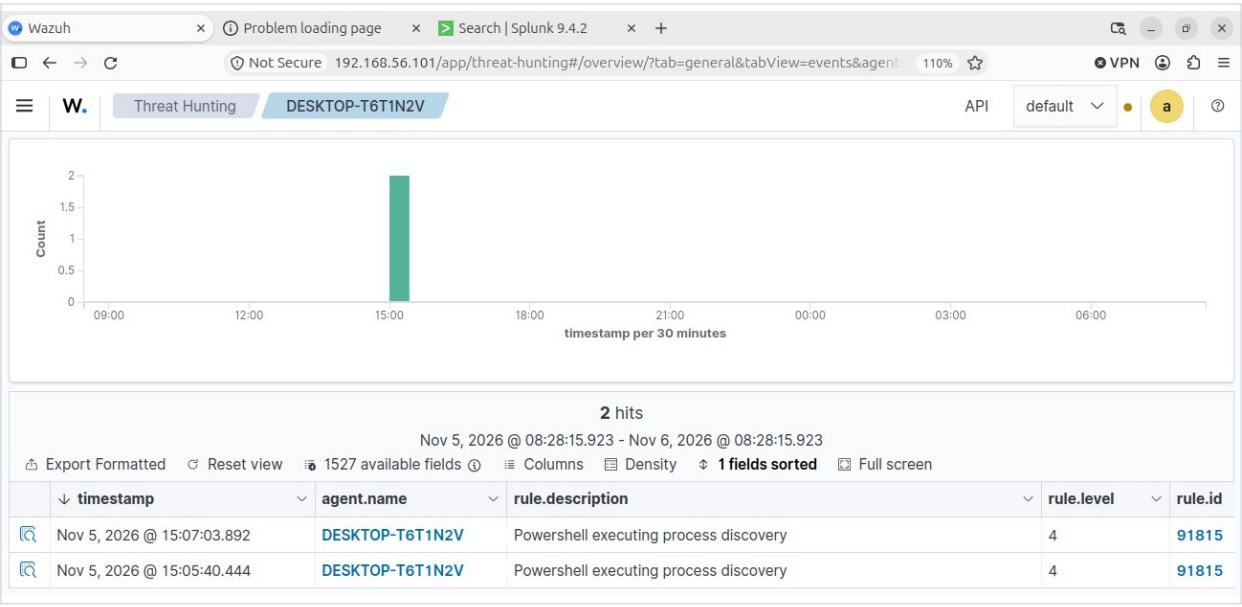


Figure 20 – Wazuh results for 4104 script-block events: rule 91815, level 4

Shows	Two hits at 15:07:03.892 and 15:05:40.444: “Powershell executing process discovery”, level 4, rule 91815.
Demonstrates	The script text that ran Get-Process was detected as process discovery, a lower-severity behaviour than the encoded command.
Limitation	Rule 91815 matches the process-listing command, not the hidden window.

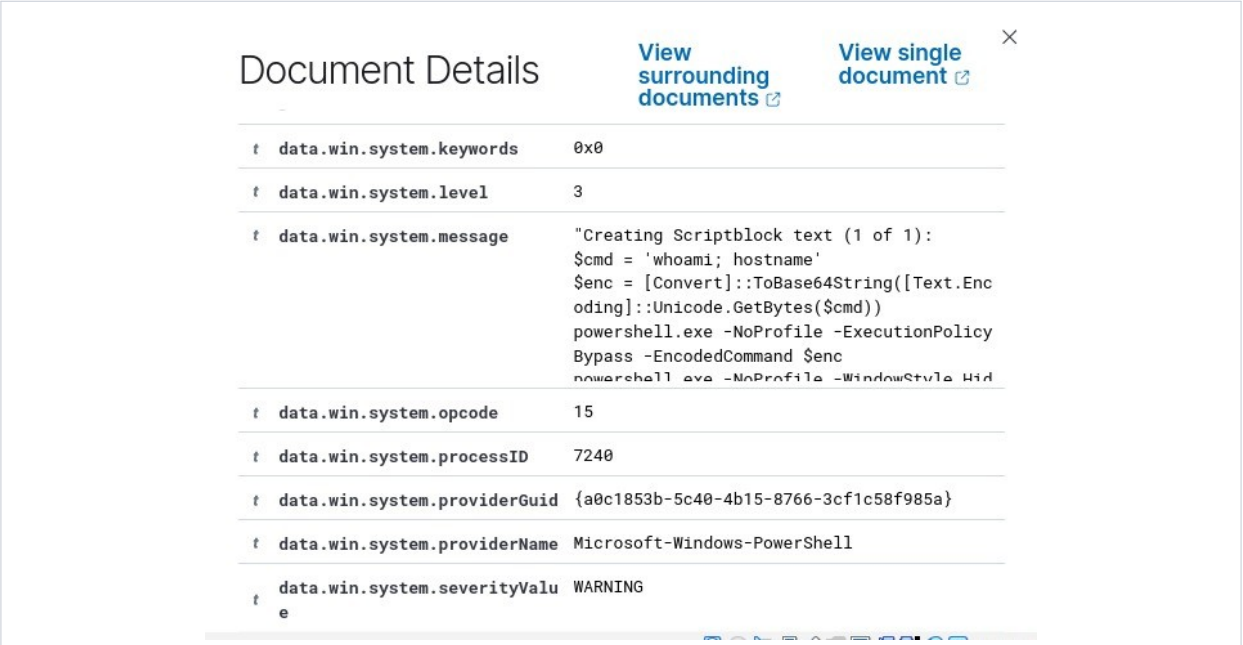


Figure 21 – The 4104 event: script text and PowerShell process ID

Shows	message “Creating Scriptblock text” with the whole test script ($cmd = ‘whoami; hostname’ ... -EncodedCommand $enc ... -WindowStyle Hidden); processID 7240; severity WARNING.
Demonstrates	The PowerShell process that logged the script (PID 7240) is the same PID the Sysmon alert lists as the parent of the encoded process. This is the cross-source correlation.
Limitation	It proves the script was *run*; it does not prove who typed it.

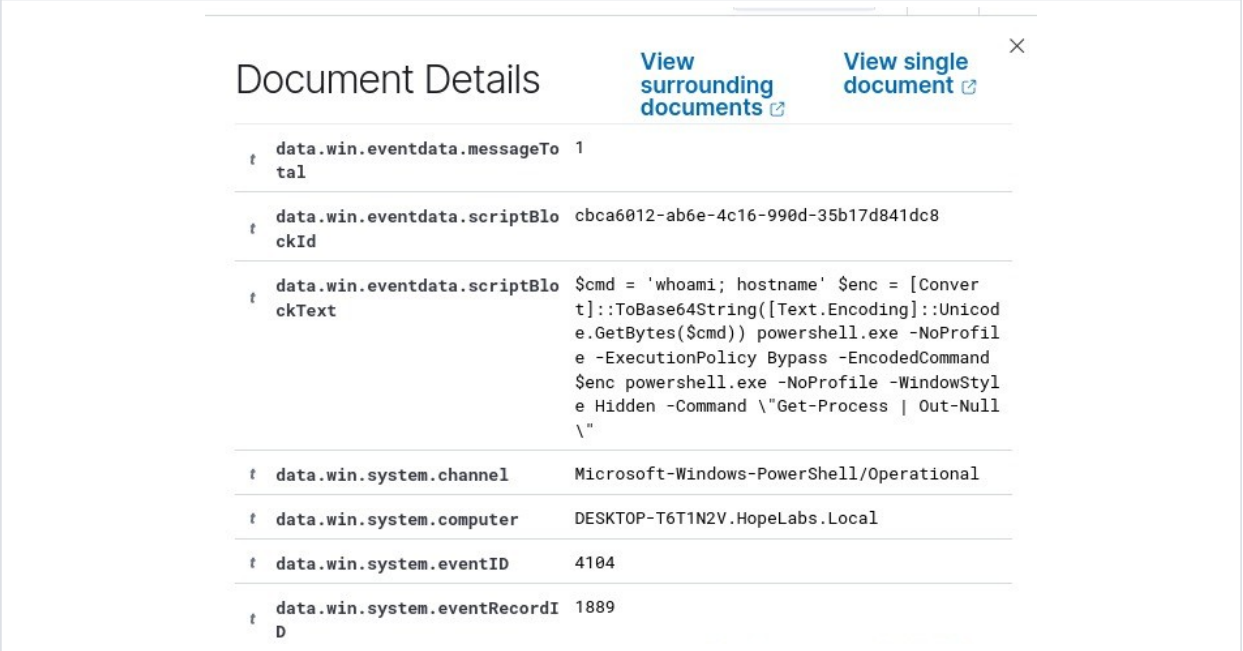


Figure 22 – The same 4104 event: record number and script-block ID

Shows	eventID 4104; eventRecordID 1889; scriptBlockId cbca6012-ab6e-4c16-990d-35b17d841dc8; channel Microsoft-Windows-PowerShell/Operational.
Demonstrates	These two identifiers let us match this event to the identical event in Splunk (Fig. 31).
Limitation	None.

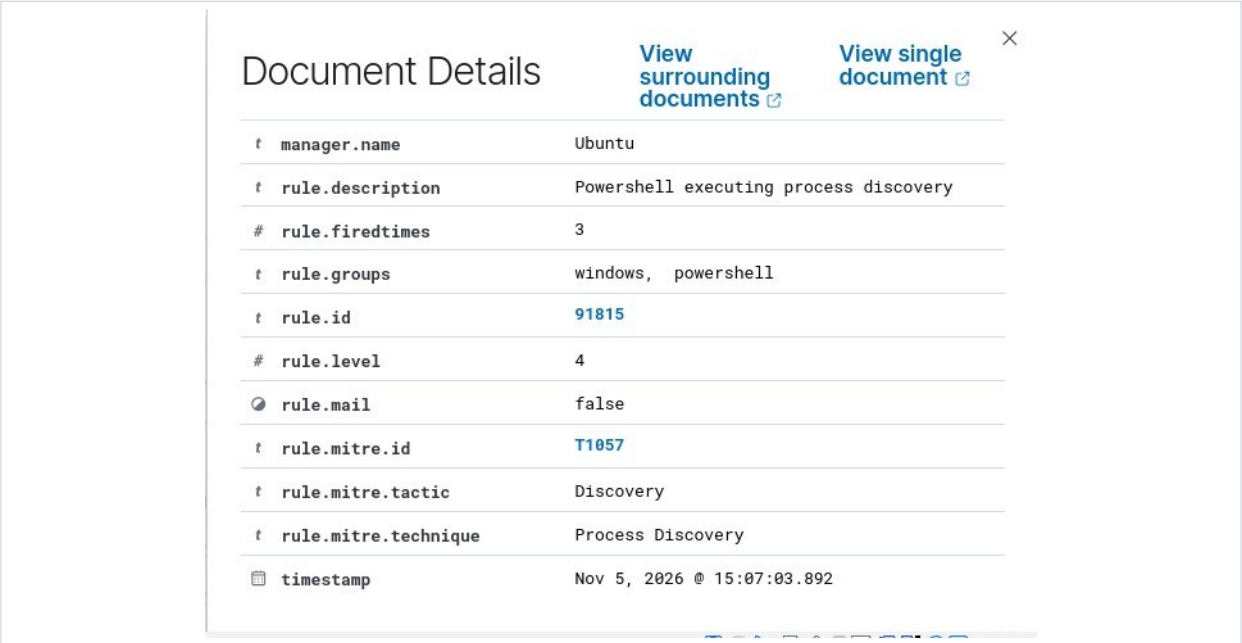


Figure 23 – Rule 91815 metadata

Shows	groups windows, powershell; level 4; MITRE T1057 (Discovery, Process Discovery); firedtimes 3.
Demonstrates	The listing of processes is mapped to a second ATT&CK technique, T1057.
Limitation	Fired three times in the period, one more than the two test runs; the third firing was not investigated.

9.3 Custom rule 100600

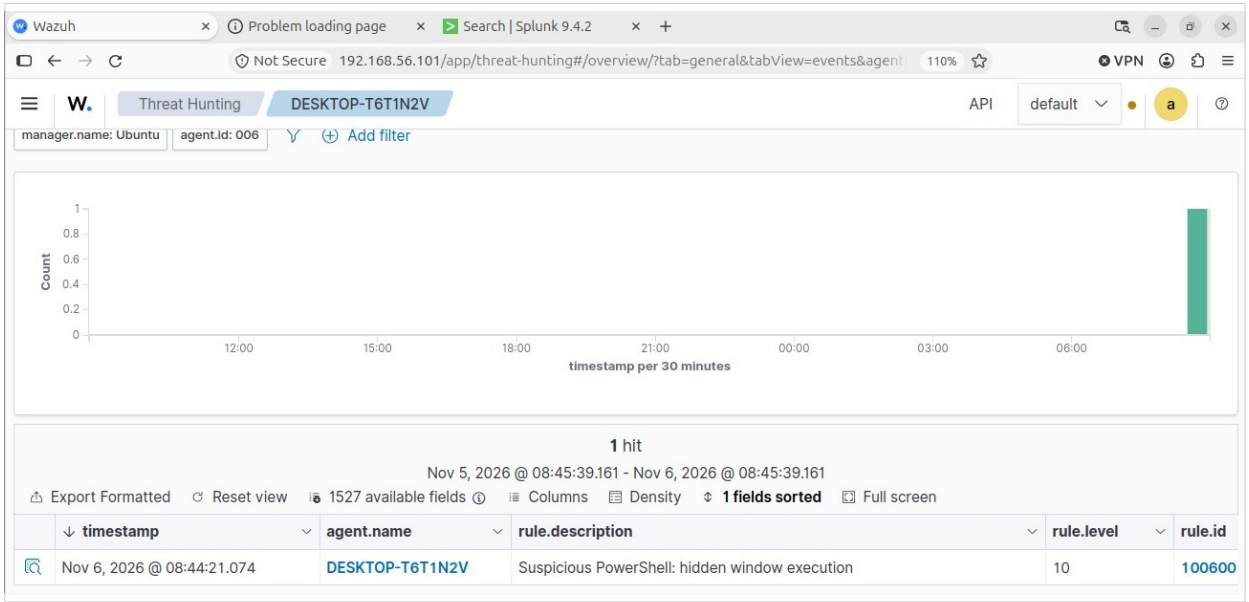


Figure 24 – Wazuh results for rule.id: 100600 (6 Nov 2026, 08:45)

Shows	One hit at Nov 6, 2026 @ 08:44:21.074; description “Suspicious PowerShell: hidden window execution”; level 10; rule 100600.
Demonstrates	Our own rule loaded and produced an alert after the manager restart.
Limitation	One firing only. The 5 Nov hidden-window runs happened *before* the rule existed, so they could not alert.

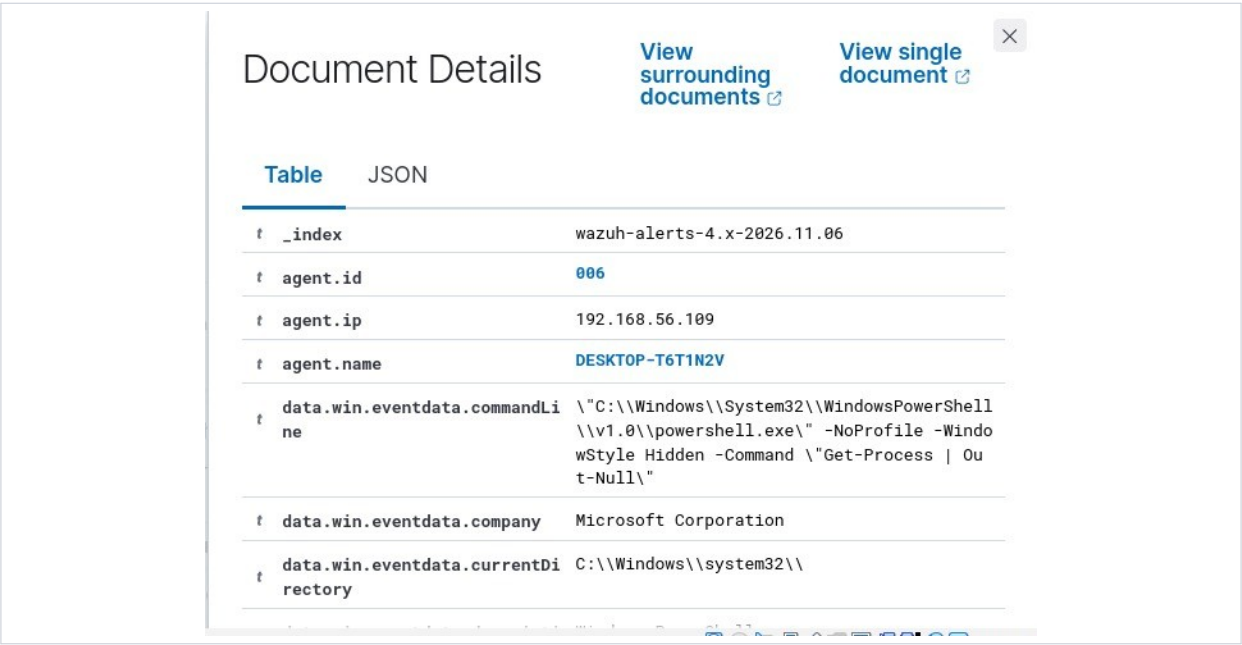


Figure 25 – The alert's command line: hidden window

Shows	commandLine "...-NoProfile -WindowStyle Hidden -Command "Get-Process | Out-Null""; index wazuh-alerts-4.x-2026.11.06; Sysmon utcTime 2026-11-06 07:44:20.967; user HOPELABS\Administrator; processId 9072; parentProcessId 4568.
Demonstrates	The rule matched the intended behaviour, not something else.
Limitation	Run by the author as a test.

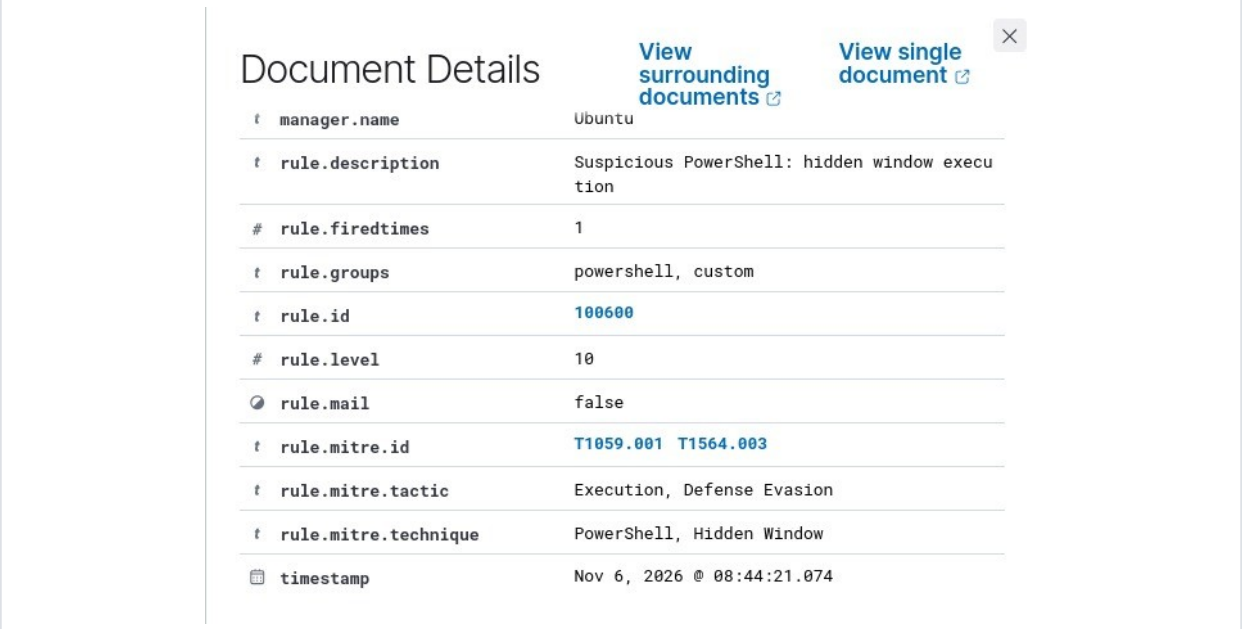


Figure 26 – Custom rule metadata and MITRE mapping

Shows	rule.groups powershell, custom; id 100600; level 10; mitre.id T1059.001 and T1564.003; tactics Execution, Defense Evasion; techniques PowerShell, Hidden Window.
Demonstrates	The MITRE deliverable is satisfied by a rule we wrote ourselves, adding Defense Evasion (T1564.003).
Limitation	None.

9.4 Splunk cross-check

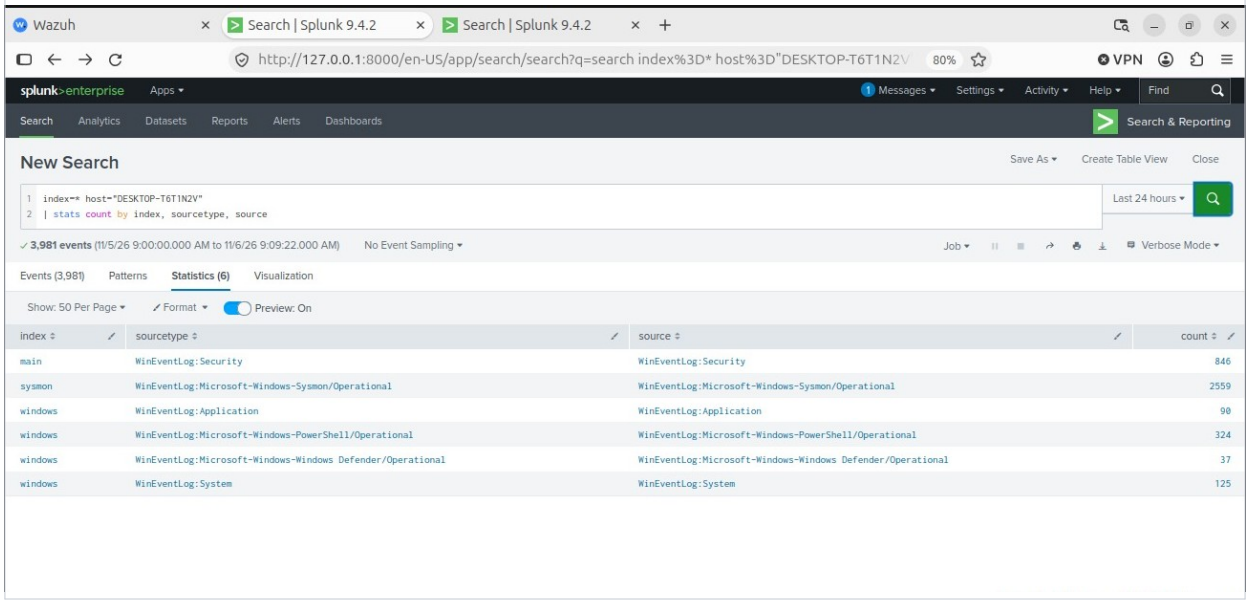


Figure 27 – Splunk: which data exists for the Windows host (6 Nov 2026, 09:28)

Shows	3,981 events in 24 hours: Sysmon 2,559 (index sysmon); PowerShell/Operational 324, System 125, Application 90, Defender 37 (index windows); Security 846 (index main).
Demonstrates	The Windows host is feeding Splunk and Sysmon and PowerShell data are present.
Limitation	Counts are a snapshot and include all activity, not just the tests.

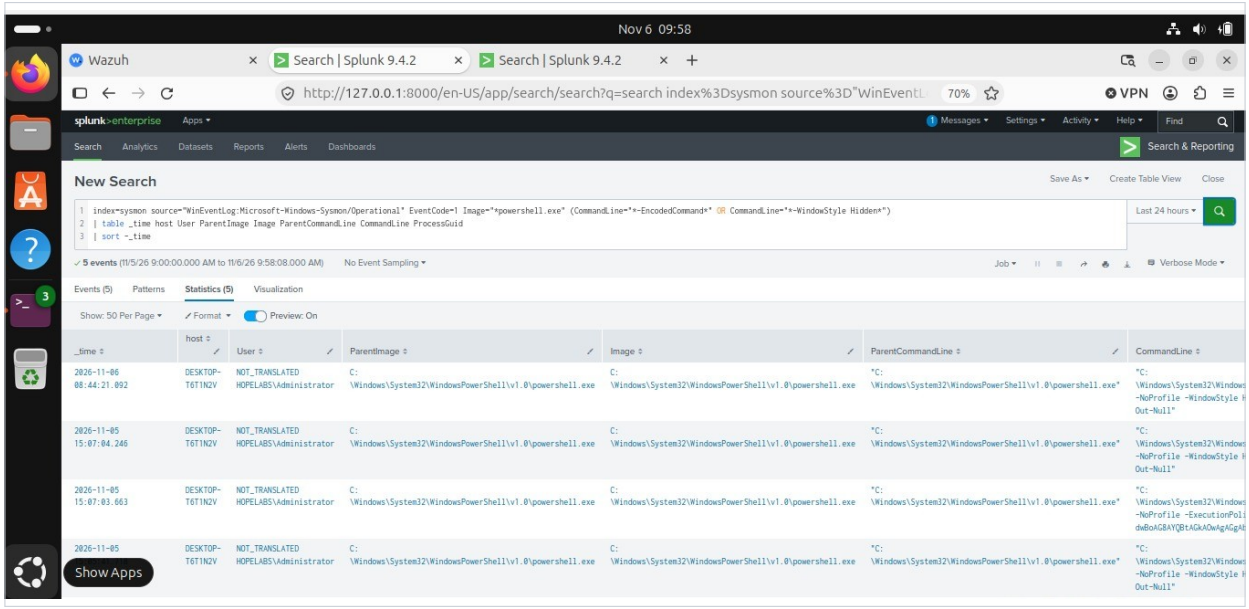


Figure 28 – Splunk: five PowerShell Sysmon events (time, host, user, parent, command line)

Shows	5 events: 08:44:21.092 on 6 Nov, and on 5 Nov 15:07:04.246, 15:07:03.663, 15:05:41.718, 15:05:40.560, each with parent powershell.exe and user HOPELABS\Administrator (displayed as NOT_TRANSLATED\...).
Demonstrates	Splunk holds the same events Wazuh alerted on, plus the 6 Nov run.
Limitation	The “NOT_TRANSLATED” prefix is a Windows add-on display quirk, not an attack sign.

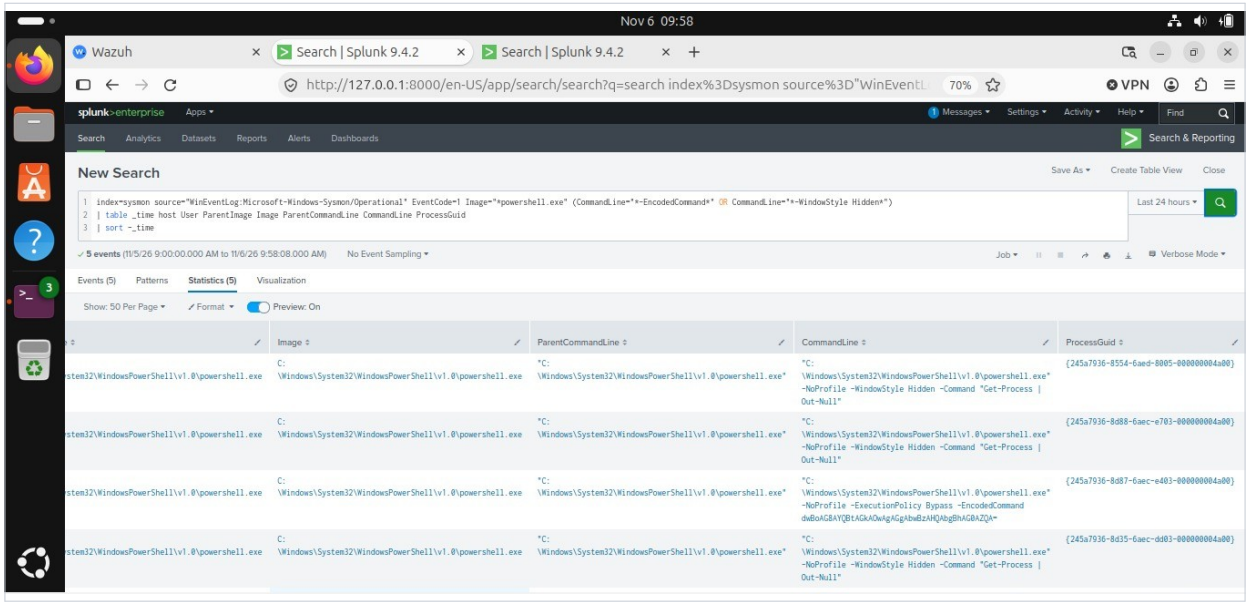


Figure 29 – The same table scrolled right: command lines and ProcessGuids

Shows	Command lines (hidden and encoded) and ProcessGuids: 8554-6aed-8005, 8d88-6aec-e703, 8d87-6aec-e403, 8d35-6aec-dd03.
Demonstrates	The ProcessGuids equal those in Wazuh and local Sysmon, so all three sources describe the same processes.
Limitation	None.

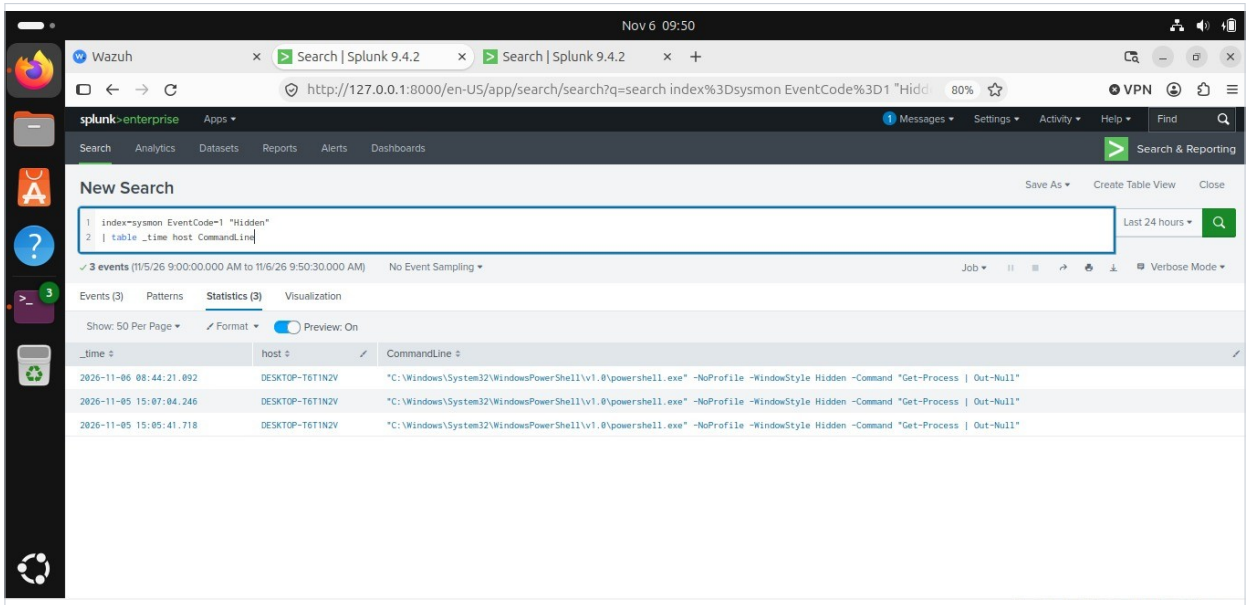


Figure 30 – Splunk: the three hidden-window runs found with a simple keyword search

Shows	Query index=sysmon EventCode=1 “Hidden” returns 3 events (08:44:21, 15:07:04, 15:05:41), each with the same command line.
Demonstrates	A broad keyword search is a reliable fallback when a field-based search misbehaves (Chapter 10).
Limitation	Keyword searches can also match unrelated text.

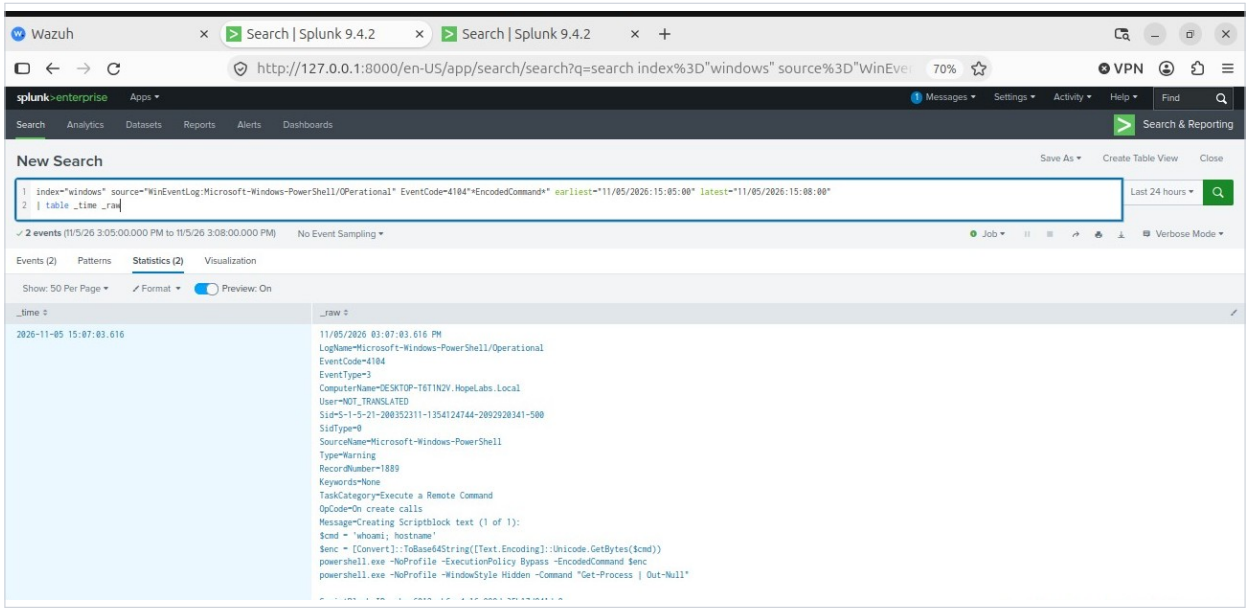


Figure 31 – Splunk raw 4104 event (15:07:03): record number 1889 and script-block ID

Shows	RecordNumber=1889; EventCode=4104; Message with the whole test script; ScriptBlock ID cbca6012-ab6e-4c16-990d-35b17d841dc8. The scrolled screenshots show a second event, record 1867, ID d72ce038-2ce1-4485-a1a5-1daed01fcaaa (15:05:40).
Demonstrates	Record number and script-block ID equal the Wazuh values (Fig. 22): the same event seen by two tools. The raw event contains no process-ID line.
Limitation	Because there is no PID in the Splunk event, the PID 7240 link rests on the Wazuh evidence.

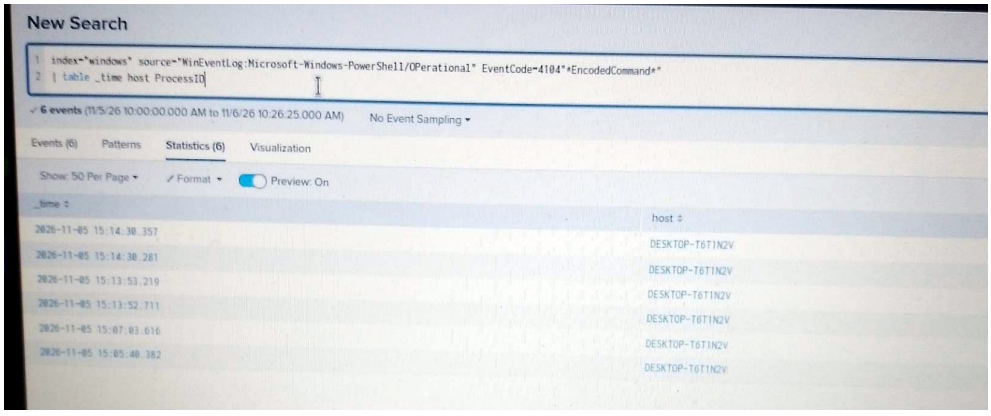


Figure 32 – Splunk 4104 search: six matching events (photo of screen, 6 Nov 10:26)

Shows	Six events on host DESKTOP-T6T1N2V: 15:05:40, 15:07:03 (the two tests) and 15:13:52, 15:13:53, 15:14:30 (twice).
Demonstrates	The test runs are present. The four later events are the author's own investigation commands, because the Sysmon filter typed at the prompt also contained the word EncodedCommand.
Limitation	Phone photo; text is legible but not as sharp as a screenshot. The ProcessID column is blank (Chapter 10, Issue 8).

Chapter 10: Troubleshooting and Root-Cause Analysis

IN PLAIN ENGLISH
Almost every real lab breaks somewhere. Writing down what broke, how we found out, and what fixed it is a core SOC skill. This table is honest about causes that were confirmed and causes that were not.

Table 15 – Issues, causes and fixes

#	Symptom	Cause	Fix and verification	Cause confirmed?
1	ModuleNames value missing in screenshots	Set-ItemProperty line was not run/shown after New-Item	Re-ran it; Get-ItemProperty showed * : * (Fig. 5)	Yes
2	Get-Service Sysmon64 says it cannot find the service	Sysmon was installed under the service name Sysmon	Searched by wildcard; found “Sysmon” Running (Fig. 7)	Yes
3	sudo: command not recognised (Windows)	Linux commands typed into Windows PowerShell	Ran them on the Ubuntu VM instead (Fig. 10)	Yes
4	Format-Table typo and an error message	Mistyped “Formal-Table”	Retyped; 4104 readback worked (Fig. 33)	Yes
5	Splunk search returned only 2 events	Typo “CommandLLine” in the second condition (Fig. 35)	Corrected spelling; later searches returned 5 events (Figs. 28–29)	Yes for the typo
6	Corrected query still returned 2 events at 09:43	Not established. Using the exact source “WinEventLog:Microsoft-Windows-Sysmon/Operational” returned all 5	Exact source string used	No (open question)
7	Splunk: “Search not executed: minimum free disk space (5000 MB) reached”	Ubuntu disk 92% full: 4.0 GB free (Fig. 37), below Splunk’s 5000 MB limit	journalctl vacuum freed 616.7 MB and the apt cache was cleaned; free space rose to 9.9 GB (Fig. 38); searches then ran	Yes (error text and df numbers)
8	Splunk 4104 query: 0 events, then a blank ProcessID column	(a) Field ScriptBlockText is not extracted in this event format, so a field-based search finds nothing; a plain keyword search worked. (b) The classic event has no process-ID field	Used keyword search (6 events, Fig. 32); correlated by record number and script-block ID (Fig. 31)	Yes for (b), confirmed by the raw event; (a) is the best explanation

10.1 Issue 4: a harmless typo

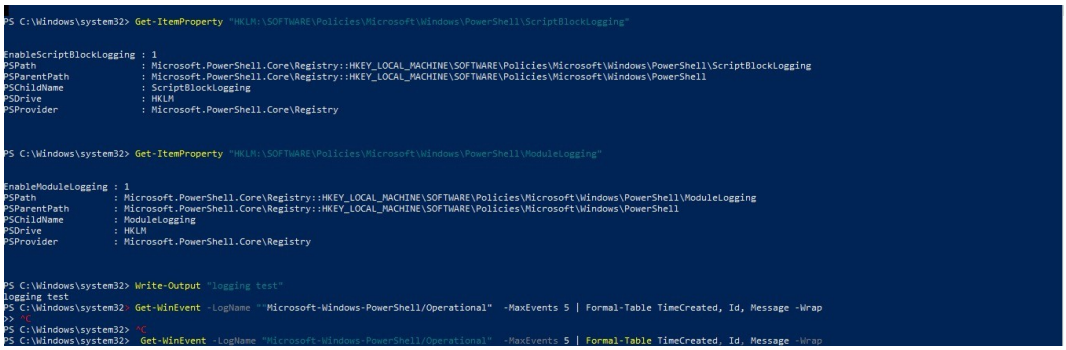


Figure 33 – Session where the typo “Formal-Table” produced an error and was retyped

Shows	Registry values read back, a test Write-Output, then a failed command followed by the corrected one.
Demonstrates	Errors were left in the evidence rather than hidden.
Limitation	None.

10.2 Issue 2: Sysmon service name

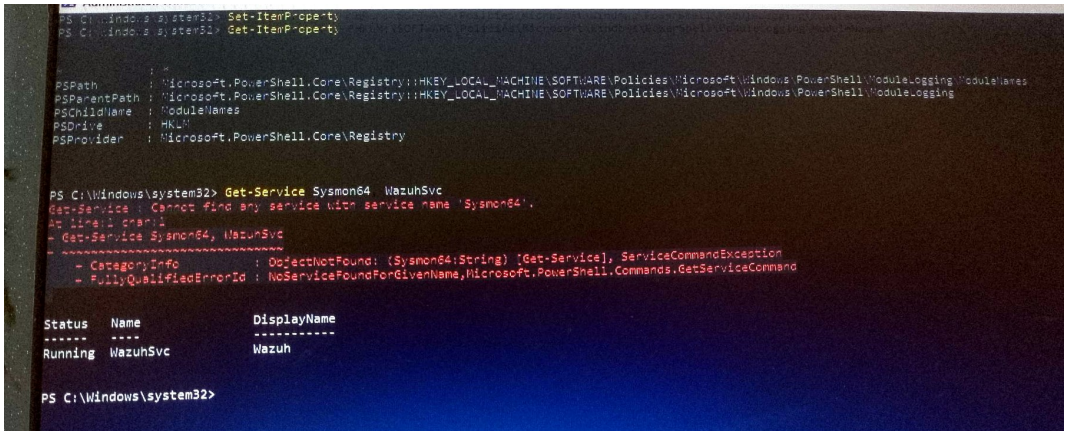


Figure 34 – Get-Service Sysmon64 fails while WazuhSvc is Running (photo of screen, 5 Nov)

Shows	Error “Cannot find any service with service name ‘Sysmon64’”; WazuhSvc Running.
Demonstrates	The agent was fine; only the Sysmon service name differed (Fig. 7).
Limitation	Photo of a monitor; some blur and colour cast.

10.3 Issue 5: the typo in Splunk

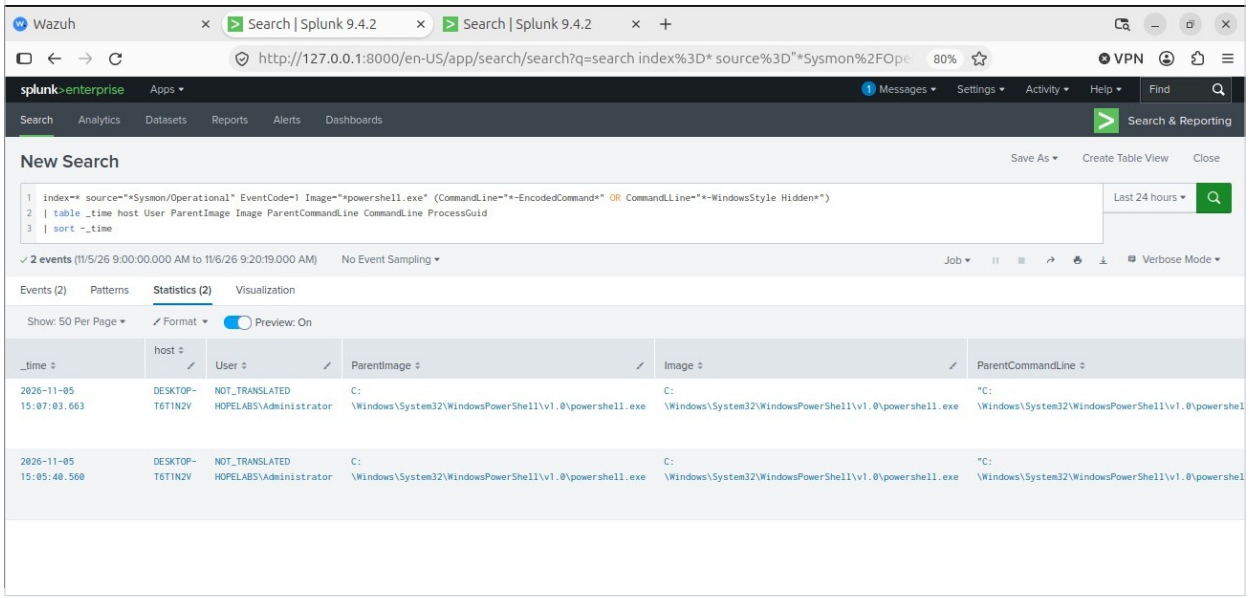


Figure 35 – Splunk query containing "CommandLLine" returns only the two encoded events

Shows	The misspelled field name in the query and 2 events.
Demonstrates	A misspelled field silently matches nothing, so the hidden-window runs were missing.
Limitation	Later screenshots (1000495638/639/653/654) show the query before it was re-run, which is why confusion continued.

10.4 Issue 7: disk full

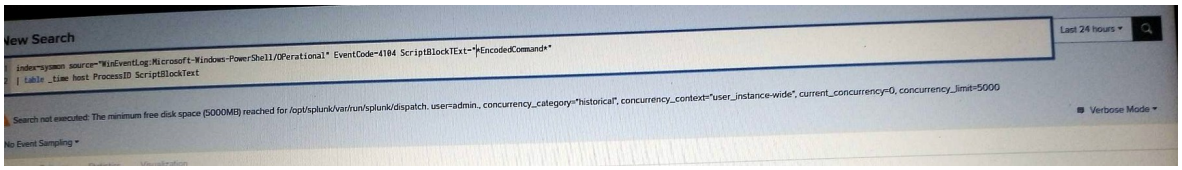


Figure 36 – Splunk refuses to run a search: minimum free disk space (5000 MB) reached (photo, 6 Nov 10:05)

Shows	Error text naming /opt/splunk/var/run/splunk/dispatch and concurrency_limit=5000.
Demonstrates	The monitoring server itself became a point of failure.
Limitation	Photo; cause shown in the next two figures.

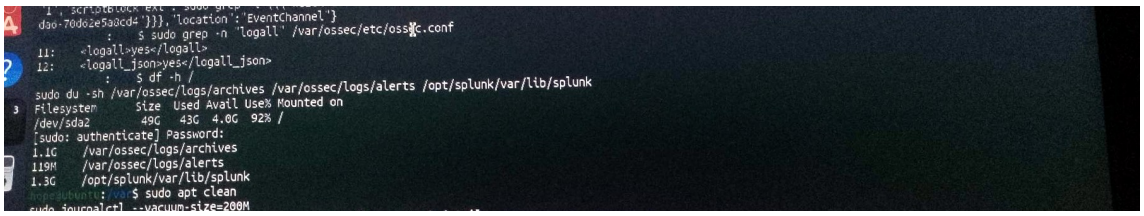


Figure 37 – df -h /: 49 GB, 43 GB used, 4.0 GB available, 92% (photo, 6 Nov 10:10)

Shows	Root disk 92% used; Wazuh archives 1.1 GB, alerts 119 MB, Splunk data 1.3 GB.
Demonstrates	These three folders explain only about 2.5 GB, so the space was used elsewhere.
Limitation	Photo; angled.

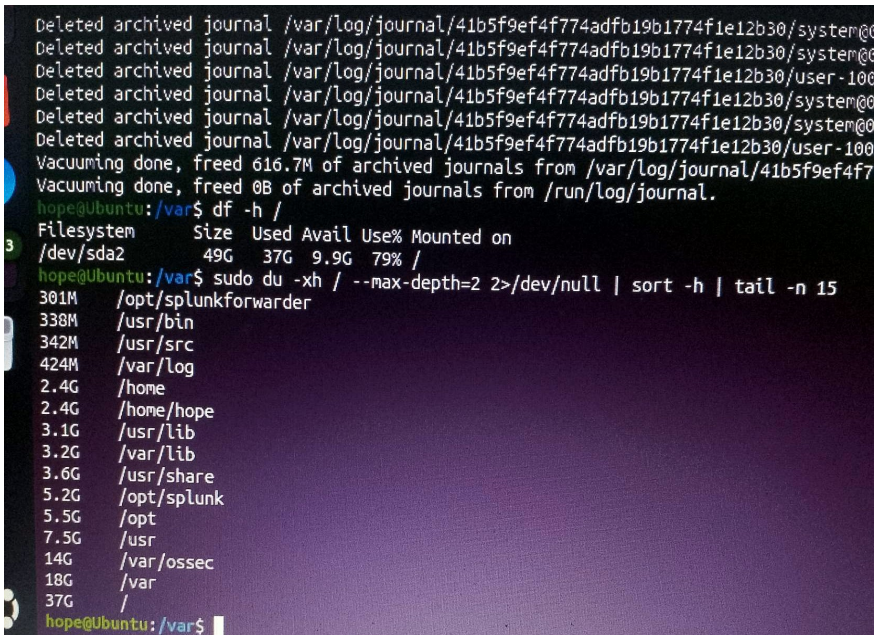


Figure 38 – After cleanup: 37 GB used, 9.9 GB available, 79%; /var/ossec is 14 GB (photo, 6 Nov)

Shows	journalctl vacuum freed 616.7 MB; free space 9.9 GB; largest folders /var 18 GB, /var/ossec 14 GB, /opt/splunk 5.2 GB.
Demonstrates	Free space is now above Splunk's 5 GB threshold.
Limitation	Free space grew by about 5.9 GB, more than the 616.7 MB the vacuum reported; the rest (apt cache, etc.) was not itemised. **Open item:** /var/ossec is 14 GB although archives and alerts measured only 1.2 GB, and was not investigated.

10.5 Issue 8: field not extracted

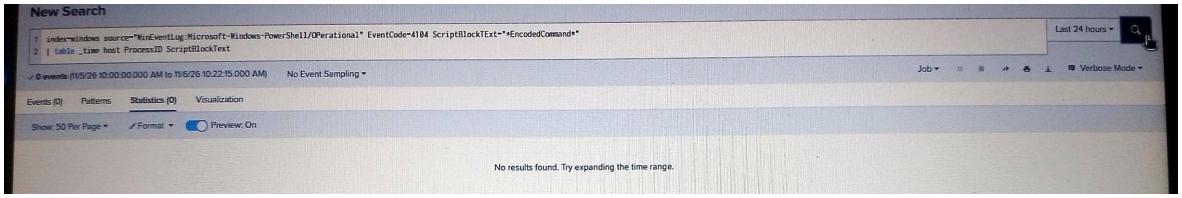


Figure 39 – A 4104 search with a field filter returns "No results found" (photo, 6 Nov 10:22)

Shows	Query filtering on ScriptBlockTExt with 0 events.
Demonstrates	A field-based filter on a field the event does not have returns nothing, even though the events exist (Fig. 32 shows 6 for a keyword search).
Limitation	The query also contained capitalisation typos (OPerational, ScriptBlockTExt). The later successful query still contained "OPerational", so capitalisation of the source was not the cause; the field filter was.



Figure 40 – Splunk table with an empty ProcessID column (photo, 6 Nov 10:27)

Shows	Six events with times and host, ProcessID column empty.
Demonstrates	Splunk does not extract a process ID from this classic event format (the raw event in Fig. 31 has none).
Limitation	Photo.

Chapter 11: Security Findings and Risk Analysis

11.1 Findings

Table 16 – Findings, evidence and confidence

ID	Finding	Evidence	Impact	Confidence
F1	Encoded PowerShell with execution-policy bypass is detected by default Wazuh rules (level 12)	Figs. 16–19	Good coverage of a common attacker technique	Confirmed
F2	Script-block logging reveals the text of code, not just that PowerShell ran	Figs. 21–22, 31	Analysts can see what was attempted	Confirmed
F3	No built-in alert named the hidden-window behaviour in the evidence; a custom rule was needed	Figs. 24–26; T10	Possible detection gap for attackers who hide windows but do not encode	Probable; not exhaustively tested
F4	Analyst commands are logged as 4104 events too	Fig. 32 (events at 15:13–15:14)	Investigation noise; reviewers must separate it	Confirmed
F5	The monitoring server's disk filled up and Splunk stopped searching	Figs. 36–38	A full disk could silently blind the SOC	Confirmed
F6	Rule 100600 will also match legitimate hidden-window use	Reasoning; Chapter 7.3	False positives; analyst fatigue	Expected; not measured

11.2 False-positive considerations (brief requirement)

- Administrators, deployment tools, scheduled tasks, installers and monitoring agents routinely run PowerShell with `-WindowStyle Hidden` or `-ExecutionPolicy Bypass` or even `-EncodedCommand`.
- Tuning options: exclude known service accounts and trusted parent processes; match known script paths; raise severity only when combined with encoding, bypass, an Office/browser parent or a download.
- The author's own test commands are the only trigger in this lab, so the false-positive rate is **unknown**.

> **CAUTION**
> The alerts in this report come from controlled tests run by the author. They are not evidence of a real intrusion.

Chapter 12: Mitigation and Remediation

Table 17 – Implemented versus recommended

Measure	Status	Benefit	Operational impact	How to verify	Residual risk
Script-block and module logging	Implemented	See what PowerShell ran	More log volume	Get-WinEvent shows 4104	Attacker with admin rights can disable logging
Collect PowerShell + Sysmon in Wazuh	Implemented	Central visibility	Agent traffic	Archive entry (Fig. 11)	Agent stopped = blind
Custom rule 100600	Implemented	Detect hidden window	False positives	rule.id: 100600 alert	Other hide methods missed
Free disk space on Ubuntu	Partly implemented	Splunk searches again	None	df -h shows >5 GB free	/var/ossec 14 GB unexplained
Expand the VM disk	Recommended	Room to grow	Downtime	df -h	None
Rotate or disable logall after the lab	Recommended	Prevent disk fill	Lose raw archive	Archive folder size stable	Less raw data
PowerShell transcription logging	Recommended	Records output too	More disk	Transcript files appear	Same as above
Constrained Language Mode, AppLocker/WDAC	Recommended	Limit what PowerShell can do	May break admin scripts	Test script is blocked	Policy bypasses
Tune rule with allow-list; add a combined encoded+bypass+hidden rule	Recommended	Fewer false positives; higher confidence	Maintenance	Benign test does not alert	Unknown benign cases
Splunk saved alert / dashboard for these searches	Recommended	Detection in a second tool	Searches use resources	Alert triggers on test	Needs tuning

Chapter 13: What I Learned

> **NOTE**
> Drafted from the actual lab record in my own voice; edit the wording as you see fit, but keep it tied to what really happened.

Concepts. I learned to separate event, log, collection, rule, alert and investigation. When something did not alert, I could ask which stage failed instead of guessing. I also learned that an encoded command is not magic: base64 decodes straight back to “whoami; hostname”, and Sysmon shows the child processes that prove what really ran.

Practical skills. I enabled PowerShell script-block and module logging through the registry, edited `ossec.conf` to collect two channels, searched the Wazuh archive, queried Sysmon with `Get-WinEvent`, and wrote search queries in Wazuh and Splunk.

Detection engineering. I wrote a custom rule, understood why it needs `if_sid` and regular expressions, and mapped it to MITRE T1059.001 and T1564.003. I also saw its limits: it only knows one way of hiding PowerShell and would fire on legitimate admin scripts.

Troubleshooting. I made real mistakes: a missing registry value, Linux commands typed on Windows, a misspelled Splunk field. The most valuable lesson was the disk-full error: the monitoring system can fail too, and “no results” is not the same as “nothing happened”.

Evidence discipline. I learned to keep failures in the record and to say “not proven”. For example, Splunk could not show the process ID, so I correlated by record number and script-block ID and said so.

Still to develop. Measuring false positives, decoding payloads systematically, tuning rules, handling disk and log retention, and investigating the unexplained 14 GB in /var/ossec.

Chapter 14: Practical Knowledge and Competencies

Table 18 – Skills mapped to evidence

Skill	Evidence	Level	Next exercise
Enable and verify Windows PowerShell logging	Figs. 4–6	Demonstrated hands-on	Add transcription logging
Configure a Wazuh agent to collect event channels	Figs. 8–10	Demonstrated hands-on	Collect Security log events
Verify log delivery at the manager	Fig. 11	Demonstrated hands-on	Use wazuh-logtest
Read Sysmon Event 1 (user, parent, command line, GUID)	Figs. 12–15	Demonstrated hands-on	Add Event 3 (network) analysis
Investigate alerts in the Wazuh dashboard	Figs. 16–26	Demonstrated hands-on	Build a saved search
Write a custom Wazuh rule with MITRE tags	Figs. 24–26	Demonstrated hands-on	Add an allow-list exception
Search in Splunk (SPL) and cross-correlate	Figs. 27–32	Demonstrated hands-on	Create a Splunk alert
Troubleshoot disk and query problems	Chapter 10	Demonstrated hands-on	Set up log rotation
Measure false positives	—	Studied only (not performed)	Run benign hidden-window scripts

Chapter 15: Reproduction Guide (for a beginner)

> **IN PLAIN ENGLISH**
> Follow the sections in order. After each one there is a “You should see” line. If you do not see it, stop and use Section 15.8 before going on. Allow 1–2 hours.

15.1 Starting state

- A Windows 10 VM, domain-joined or standalone, with the Wazuh agent installed and active.
- Sysmon installed (service name may be Sysmon or Sysmon64).
- An Ubuntu VM with Wazuh manager and dashboard running. Take a snapshot of both VMs now.
- **At least 10 GB free** on the Ubuntu root disk: run `df -h /`.

15.2 Enable PowerShell logging (Windows 10, PowerShell as Administrator)

Right-click Start → *Windows PowerShell (Admin)*. Run the six commands in Step 1 of Chapter 6, then verify:

```
Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"
Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging\ModuleNames"
```

You should see: EnableScriptBlockLogging : 1 and a property named * with the value *.

15.3 Check Sysmon and the agent (Windows 10)

```
Get-Service WazuhSvc
Get-Service | Where-Object { $_.Name -like "*sysmon*" }
Get-WinEvent -ListLog "Microsoft-Windows-Sysmon/Operational"
```

You should see: both services Running, and the Sysmon log listed with a record count above 0. If Sysmon is missing, install it (with a process-creation configuration) before continuing.

15.4 Collect the logs (Windows 10)

1. Run `notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"` as Administrator.
2. Scroll near the bottom (above the closing). Use Ctrl+F to see which blocks exist.
3. Add the two blocks from Step 3 of Chapter 6, but only the ones that are missing. Save with Ctrl+S.
4. Restart the agent: `Restart-Service WazuhSvc`

```
Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "PowerShell/Operational","Sysmon/Opera
```

You should see: two matching lines.

15.5 Optional: archive logging (Ubuntu VM)

Archives record every event, which makes collection easy to prove but uses disk. In `/var/ossec/etc/ossec.conf` make sure `<logall_json>yes</logall_json>` is set, then restart:

```
sudo systemctl restart wazuh-manager
sudo grep -n "logall" /var/ossec/etc/ossec.conf
```

> **CAUTION**
> Turn archive logging off when the lab is finished, or the disk will fill (Chapter 10, Issue 7).

15.6 Generate the test events (Windows 10)

Run the four lines in Step 5 of Chapter 6. **You should see:** the first command prints your user and host name; nothing is displayed for the second. Then prove delivery on Ubuntu:

```
sudo grep -i "EncodedCommand" /var/ossec/logs/archives/archives.json | tail -n 1
```

15.7 Find the alerts and add your own rule

1. Open the Wazuh dashboard → *Threat Hunting* → your Windows agent → *Events*. Set the time range to the last 24 hours.
2. Search: `data.win.eventdata.commandLine: *EncodedCommand*`. **You should see** rule 92057 (level 12).
3. On Ubuntu check that the rule ID is free: `sudo grep -rn "100600" /var/ossec/etc/rules/` (empty output is good).
4. Edit `sudo nano /var/ossec/etc/rules/local_rules.xml` and paste the rule from Section 7.2 at the bottom. Save (Ctrl+O, Enter, Ctrl+X).
5. Check syntax: `sudo /var/ossec/bin/wazuh-analysisd -t` (no errors), then `sudo systemctl restart wazuh-manager`.
6. On Windows, run the hidden-window command again. In the dashboard search `rule.id: 100600`. **You should see** one alert with level 10 and MITRE T1059.001 / T1564.003.

Optional Splunk check

In Splunk Search, set Last 24 hours and run the queries in Section 7.4 and Appendix C. If you get a disk error, free space first (Section 15.8).

15.8 Troubleshooting quick guide

Table 19 – If you do not see the expected result

Symptom	Likely cause	What to do
Get-Service says service not found	Different service name	Search with a wildcard (Section 15.3)
No PowerShell event in the archive	Channel missing, agent not restarted, archives off	Check ossec.conf lines; restart agent; check logall_json
No alert for rule 100600	Syntax error, manager not restarted, or the command ran before the rule existed	Run the syntax check; restart manager; run the command again
Splunk “minimum free disk space” error	Less than 5 GB free	sudo journalctl --vacuum-size=200M; sudo apt clean; df -h /; expand the disk if needed
Splunk field search returns nothing	Field not extracted in this event format	Use a plain keyword search in quotes
sudo: command not recognised	You are in Windows PowerShell	Run Linux commands on the Ubuntu VM

15.9 Cleanup and rollback

Windows 10, elevated PowerShell (only if you want to undo the lab)

```
Remove-Item "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Recurse
Remove-Item "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging" -Recurse
Restart-Service WazuhSvc
```

Also remove the PowerShell block from `ossec.conf`, delete rule 100600 from `local_rules.xml`, set `logall_json` to no, restart the manager, or simply revert your snapshots.

Chapter 16: Limitations and Future Improvements

- **One host, benign tests.** All detections come from the author’s own harmless commands on one machine.
- **One custom-rule firing.** The 5 Nov hidden-window runs predate the rule; only the 6 Nov run triggered it.
- **No false-positive measurement.** No benign hidden-window script was run.
- **Module-logging events (4103) not inspected.** Only the registry setting was checked.
- **Rule file and syntax-check not captured** in a screenshot.
- **Splunk has no process ID** for this event format; correlation relies on record number and script-block ID plus Wazuh.
- **Some evidence is phone photos** of the monitor (disk, Splunk), with some blur and colour cast; the key detection evidence is screenshots.
- **Unexplained disk use:** /var/ossec is 14 GB; the cause is unknown.
- **Open question:** why the corrected Splunk query at 09:43 still returned 2 events (Issue 6).
- **The third firing** of rule 91815 was not investigated.

Future improvements

- Run benign hidden-window and encoded scripts to measure false positives, then tune.
- Add a combined rule (encoded + bypass + hidden) at higher severity.
- Decode and analyse the payload of the encoded process’s own 4104 event.
- Add Sysmon Event 3 (network) and file-creation analysis.
- Add PowerShell transcription; rotate or disable archive logs; expand the Ubuntu disk.
- Create a Splunk alert and a dashboard for these searches.

Chapter 17: Conclusion

Day 6 achieved its goal: PowerShell activity on a Windows 10 VM became visible, searchable and alertable. Logging was enabled and verified; the Wazuh agent delivered PowerShell and Sysmon events to the manager; built-in rules 92057 (level 12) and 91815 (level 4) fired on the test activity; a custom rule, 100600, fired on a hidden-window command and carries MITRE T1059.001 and T1564.003; and Splunk independently showed the same five Sysmon events and the same script block.

The strongest evidence is the cross-source agreement: matching ProcessGuids across Sysmon, Wazuh and Splunk, and matching record number 1889 and script-block ID in Wazuh and Splunk. The weakest areas are the missing false-positive measurement and the single firing of the custom rule. The disk-full episode is a useful reminder that detection depends on healthy infrastructure.

Next steps: measure and tune false positives, add network and file context, formalise log retention, and continue to Day 7 of the program.

Appendix A: Evidence Register

Original photos and screenshots were left unchanged. The figures in this report are cropped presentation copies. Phone photos also had mild contrast and sharpness adjustment. No content was edited, redrawn or removed. Original file names are listed so each figure can be verified.

Table 20 – Figure-to-original evidence register

Fig.	Description	Original file(s)
1	The Day 6 lab brief (source of the tasks, deliverables and analyst questions; also the layout reference for this report)	1000492138.jpg
2	The six stages from event to investigation. A problem at any stage looks like “no alert”.	Drawn for this report (not a screenshot)
3	Laboratory architecture and data flow (drawn from the IP addresses, agent ID and index names seen in the evidence)	Drawn for this report (not a screenshot)
4	The registry keys being created (Windows 10, 3 Nov 2026, 12:57 PM)	1000492405.jpg
5	ModuleNames set to “*” and read back, completing module logging	1000494629.jpg
6	Event ID 4104 entries showing the commands just typed (12:49–12:55 PM)	1000492408.jpg (also 1000492404.jpg, 1000492407.jpg)
7	Sysmon found running under the service name “Sysmon”	1000494632.jpg
8	ossec.conf in Notepad: the existing Sysmon block (5 Nov 2026, 2:48 PM)	1000494635.jpg
9	Agent restarted and reported Running (2:55 PM)	1000494641.jpg
10	Select-String shows both channels at lines 50 and 65 (2:59 PM). The two red sudo errors are explained in Chapter 10, Issue 3.	1000494644.jpg
11	Archive entry for agent DESKTOP-T6T1N2V, channel PowerShell/Operational, Event ID 4104; logall and logall_json set to yes	1000494647.jpg
12	Sysmon Event 1: encoded PowerShell (PID 9492) started at 3:07:03 PM by parent PID 7240	1000494684.jpg
13	Sysmon Event 1: the earlier encoded run (PID 6724) at 3:05:40 PM, parent PID 7672	1000494696.jpg
14	Child processes whoami.exe and HOSTNAME.EXE; their ParentCommandLine is the encoded command	1000494681.jpg
15	Sysmon Event 1: hidden-window PowerShell (PID 2444) at 3:07:04 PM, parent PID 7240	1000494675.jpg
16	Wazuh Threat Hunting results for the encoded-command query: two level-12 alerts (6 Nov 2026)	1000495403.jpg (also 1000495402.jpg)
17	Alert fields: the command line as seen by Wazuh	1000495404.jpg
18	Alert fields: parent process and process IDs	1000495408.jpg (also 1000495406.jpg, 1000495407.jpg)
19	Alert fields: rule metadata for 92057	1000495418.jpg (also 1000495419.jpg)
20	Wazuh results for 4104 script-block events: rule 91815, level 4	1000495475.jpg (also 1000495474.jpg)
21	The 4104 event: script text and PowerShell process ID	1000495479.jpg
22	The same 4104 event: record number and script-block ID	1000495477.jpg (also 1000495476.jpg, 1000495478.jpg)
23	Rule 91815 metadata	1000495482.jpg
24	Wazuh results for rule.id: 100600 (6 Nov 2026, 08:45)	1000495553.jpg (also 1000495550.jpg)

Fig.	Description	Original file(s)
25	The alert's command line: hidden window	1000495554.jpg (also 1000495555–569.jpg)
26	Custom rule metadata and MITRE mapping	1000495574.jpg (also 1000495573.jpg)
27	Splunk: which data exists for the Windows host (6 Nov 2026, 09:28)	1000495625.jpg
28	Splunk: five PowerShell Sysmon events (time, host, user, parent, command line)	1000495702.jpg (also 1000495699.jpg)
29	The same table scrolled right: command lines and ProcessGuids	1000495705.jpg
30	Splunk: the three hidden-window runs found with a simple keyword search	1000495699.jpg
31	Splunk raw 4104 event (15:07:03): record number 1889 and script-block ID	1000495774.jpg (also 1000495773.jpg, 1000495775.jpg)
32	Splunk 4104 search: six matching events (photo of screen, 6 Nov 10:26)	20261106_10263291975682901009908 85.jpg
33	Session where the typo "Formal-Table" produced an error and was retyped	1000492407.jpg
34	Get-Service Sysmon64 fails while WazuhSvc is Running (photo of screen, 5 Nov)	20261105_14395228696608477755586 06.jpg
35	Splunk query containing "CommandLLine" returns only the two encoded events	1000495621.jpg (also 1000495622.jpg)
36	Splunk refuses to run a search: minimum free disk space (5000 MB) reached (photo, 6 Nov 10:05)	20261106_10053548570010358145431 7.jpg
37	df -h /: 49 GB, 43 GB used, 4.0 GB available, 92% (photo, 6 Nov 10:10)	20261106_10095689922436479990938 74.jpg (also 20261106_10101977510288111801824 14.jpg, 20261106_10103881332277425439784 28.jpg)
38	After cleanup: 37 GB used, 9.9 GB available, 79%; /var/ossec is 14 GB (photo, 6 Nov)	20261106_10125452903249570013813 26.jpg
39	A 4104 search with a field filter returns "No results found" (photo, 6 Nov 10:22)	20261106_10221730896455768236510 7.jpg
40	Splunk table with an empty ProcessID column (photo, 6 Nov 10:27)	20261106_10275834800899743177091 9.jpg

Appendix B: Commands Used

```
# Windows 10 (PowerShell as Administrator)
Get-Service Sysmon64, WazuhSvc
Get-Service | Where-Object { $_.Name -like "*sysmon*" -or $_.DisplayName -like "*sysmon*" }
notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"
Restart-Service WazuhSvc
Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "PowerShell/Operational","Sysmon/Opera
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1} -MaxEvents 200 |
  Where-Object { $_.Message -match 'EncodedCommand|WindowStyle Hidden' } | Format-List TimeCreated, Message

# Ubuntu VM
sudo grep -i "wazuh collection test day6" /var/ossec/logs/archives/archives.json | tail -n 1
sudo grep -n "logall" /var/ossec/etc/ossec.conf
df -h /
sudo du -sh /var/ossec/logs/archives /var/ossec/logs/alerts /opt/splunk/var/lib/splunk
sudo apt clean
sudo journalctl --vacuum-size=200M
sudo du -xh / --max-depth=2 2>/dev/null | sort -h | tail -n 15
```

Appendix C: Splunk Queries

```
index=* host="DESKTOP-T6T1N2V" | stats count by index, sourcetype, source

index=sysmon source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 Image="*powershell.exe"
  (CommandLine="*-EncodedCommand*" OR CommandLine="*-WindowStyle Hidden*")
  | table _time host User ParentImage Image ParentCommandLine CommandLine ProcessGuid | sort -_time

index=sysmon EventCode=1 "Hidden" | table _time host CommandLine

index="windows" source="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104 "*EncodedCommand*"
  | table _time host

index=windows source="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104 "EncodedCommand"
  earliest="11/05/2026:15:05:00" latest="11/05/2026:15:08:00" | table _time _raw
```

Appendix D: Glossary in Plain Language

Table 21 – Plain-language glossary

Term	Plain meaning
Agent	A small program that watches a computer and reports to a server.
Alert	A warning the SIEM creates when a rule matches an event.
Base64	A way of writing data as letters and digits. It hides text from a quick glance but is easily reversed.
Event	One recorded thing that happened.
Hash	A fingerprint of a file.
Index (Splunk)	A labelled storage area for events.
Level (Wazuh)	How serious an alert is, from 0 to 15.
Log	A diary a computer keeps of what it did.
Parent process	The program that started another program.
Rule	A pattern the SIEM looks for in events.
Registry	Windows' central settings database.
Script block	A chunk of PowerShell code.
SIEM	Software that collects logs and raises alerts.
Snapshot	A saved point-in-time copy of a VM to roll back to.

Appendix E: Integrity and Redaction Note

- Originals are preserved unchanged; this report uses cropped copies. Phone photos received mild auto-contrast and sharpening only.
- No values, errors, timestamps, IDs or alerts were altered, added or removed; errors (typos, the disk-full message) are shown on purpose.
- No redaction was applied. Visible items include private lab IP addresses (192.168.56.x), host and domain names, user names, and a Windows security identifier in one Splunk raw event. These are lab-only, but consider redacting them before publishing publicly.
- Figure 2 and Figure 3 style diagrams were drawn for this report from facts visible in the evidence; they are not screenshots.